

**Project Assignment
of
Advanced Data Management:
By**

Fatemeh Ebadi

**Professor:
Andrea Campagner**

May 2026

Table of Contents

1	Project Description	5
1.1	Different types of data.....	5
1.2	Part 1- Comparing the Relational and Document Models.....	7
1.2.1	Data Modeling	7
1.2.1.1	Design a relational schema in an RDBMS.....	7
1.2.1.2	Design a document schema in a document-oriented NoSQL DBMS	12
1.2.2	Query Implementation	14
1.2.2.1	Generating simulated data to populate schemas	14
1.2.2.1.1	Implement Queries in an RDBMS.....	15
1.2.2.1.2A	document-oriented NoSQL DBMS.....	24
1.2.3	Schema Evolution.....	38
1.2.3.1	Schema Evolution in RDBS	38
1.2.3.2	Schema Evolution in the Document-based.....	40
1.2.4	Spark-based Implementation.....	42
1.3	Part 2: Graph Model.....	49
1.3.1	Query Implementation	49
1.3.1.1	Design a graph-based data model	49
1.3.1.2	Spark-based Implementation	67
1.3.2	Partitioning and Replication Plan	72
1.3.2.1	Partitioning.....	72
1.3.2.2	Replication	74

Figures:

Figure 1-Identifying the entities of systems.....	5
Figure 2-Showing the existence identities.....	7
Figure 3-The Relational Data Base Scheme and the relation between different Identities.....	8
Figure 4-Creating DataBase in RDBS	8
Figure 5-The scripts of Table creation in RDBS.....	10
Figure 6-The Schema of the Document-Based Database	12
Figure 7-Summarize of prtitioning question	72

Tables:

Table 1-Database Creation Script (RDBMS).....	9
Table 2-The scripts of Table creation in RDBS	11
Table 3-The scripts of the creation the Tables in document -based DB.....	13
Table 4-Python Script for Random Data Generation in RDBMS	16
Table 5-The times of queries implementation.....	23
Table 6- Python Script for Random Data Generation in document-based DB	25
Table 7-The times of queries implementation id Document-based DB	36
Table 8-The script of insert additional Event Value	38
Table 9-Comparison between RDBMS and MongoDB regarding to adding a new column	40
Table 10-Python Script for Searching and Assigning Battery Values (0–100)	41
Table 11-Query 2 Spark Execution in the Document-Based Model (No Timer)	43
Table 12-Query 2 Spark Execution in the Document-Based Model (With Timer).....	44
Table 13-The times of queries implementation id Document-based DB	47
Table 14-Comparison between the result of queries due to MongoDB and Spark implementations.....	48
Table 15-Scripts of designing a Graph-based data model	49
Table 16-Python Script for Random Data Generation in Graph DB without and with Timer	54
Table 17-The times of queries implementation id Graph-based DB	66
Table 18-Spark Implementation for some queries in the Document-Based Database	67
Table 19-The times of queries implementation id Graph-based DB and Spark	69
Table 20-Performance and Scalability Comparison Between Neo4j and Apache Spark Queries	71

1 Project Description

The aim of the project is to implement the data management component of a city mobility platform, aimed at managing the short-term rental of shared electric vehicles across different cities in Italy.

1.1 Different types of data

We have to identify the people who are involved with this system and will be working with it? This leads us to define various types of data which is included: Users, Trips, Stations, Events generated during trips as it is shown in the Figure 1.

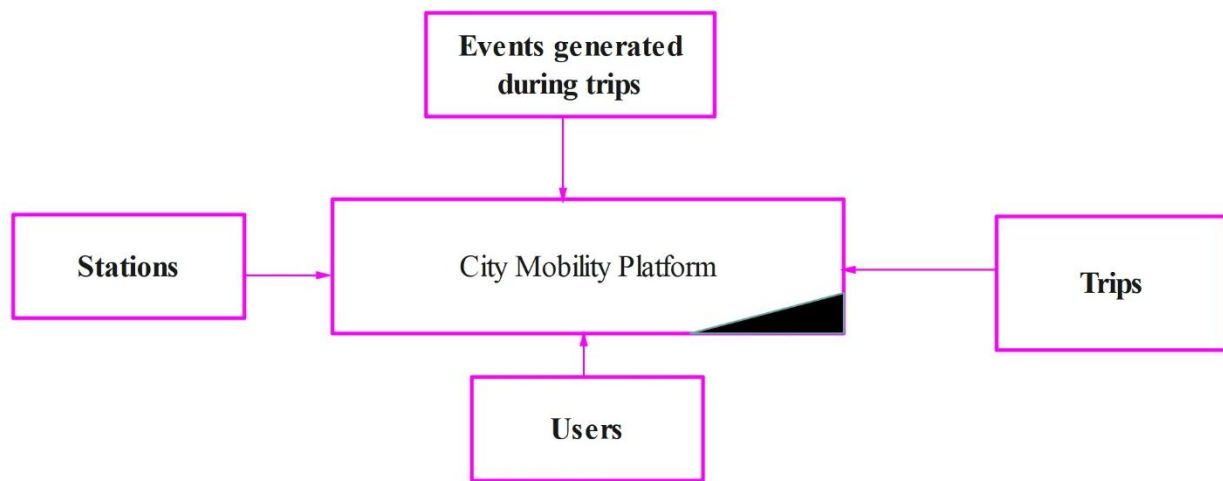


Figure 1-Identifying the entities of systems.

The detailed information is described in the below.

a. User:

The '**Users**' entity maintains comprehensive profiles for all platform members, capturing essential demographics such as a name, surname, birthdate, and country of origin.

b. Station:

'**Stations**' represent points of interest tracked by the mobility platform, in which users can deposit or take some vehicles. Each station has a name, the specific city, the location and the maximum capacity of vehicles which can be stored;

c. Trip:

Each trip is associated with a user, one start and end station, the start and end time, and a total cost.

d. **Event:**

The system records every event that occurs along the way as an event and it can be zero or more. Here, there are three categories for this information to make everything completely obvious.

➤ **Status Information (Status)**

These are not errors; they are normal system reports such as:

- ✓ **GPS:** It periodically reports the user's current location. (For example: "The user is currently on X Street.")
- ✓ **BATTERY:** It reports the current battery level. (For example: "The battery level is currently at 80%.")

This information helps the system determine whether the device needs charging or not.

➤ **Errors or Faults (Errors/Alerts)**

This section refers to situations where something goes wrong:

- ✓ **ERROR:** If the motor overheats, the vehicle fails to unlock, or the device loses its internet connection, an event of type **ERROR** is logged.
- ✓ **BATTERY (Critical):** If the battery level drops below 5%, an event is recorded, which may cause the trip to stop.

➤ **Timing Events and Delays (Delays)**

- ✓ **DELAY:** If the user parks the vehicle outside the designated parking zone, or if they get stuck in unusually heavy traffic that the system detects as abnormal, a **DELAY** event is recorded.

Moreover, each event is associated with a trip, a timestamp (describing when the event happened), the type of event (**GPS**, **BATTERY**, **ERROR**, **DELAY**) and a value or information of event type.

I tried to show the different entities in the Figure 2.

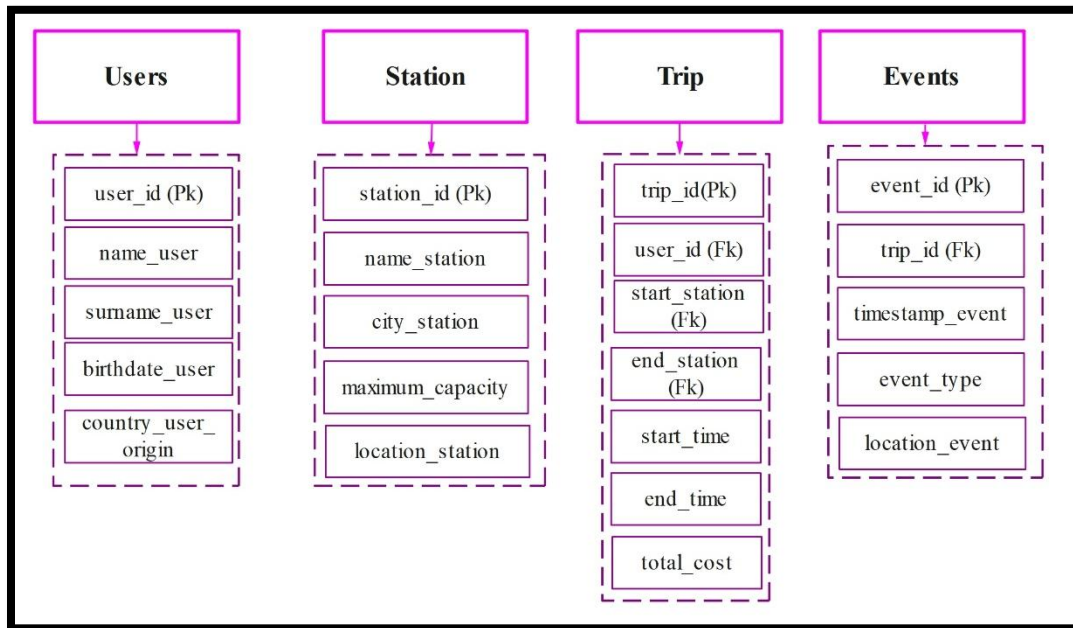


Figure 2-Showing the existence identities.

1.2 Part 1- Comparing the Relational and Document Models

1.2.1 Data Modeling

1.2.1.1 Design a relational schema in an RDBMS

The diagram illustrates the structure of the database and the relationships between its four main tables (Users, Trip, Station, Events). The users table stores personal information about each user, while the station table contains details about every station in the system. The trip table records individual trips, linking each trip to the user who made it and to the start and end stations through foreign keys. Finally, the events table logs specific events that occur during a trip, such as timestamps or status changes, and is connected to the trip table. Together, these relationships create a coherent data model that allows the system to track users, stations, trips, and all related events in an organized and consistent way.

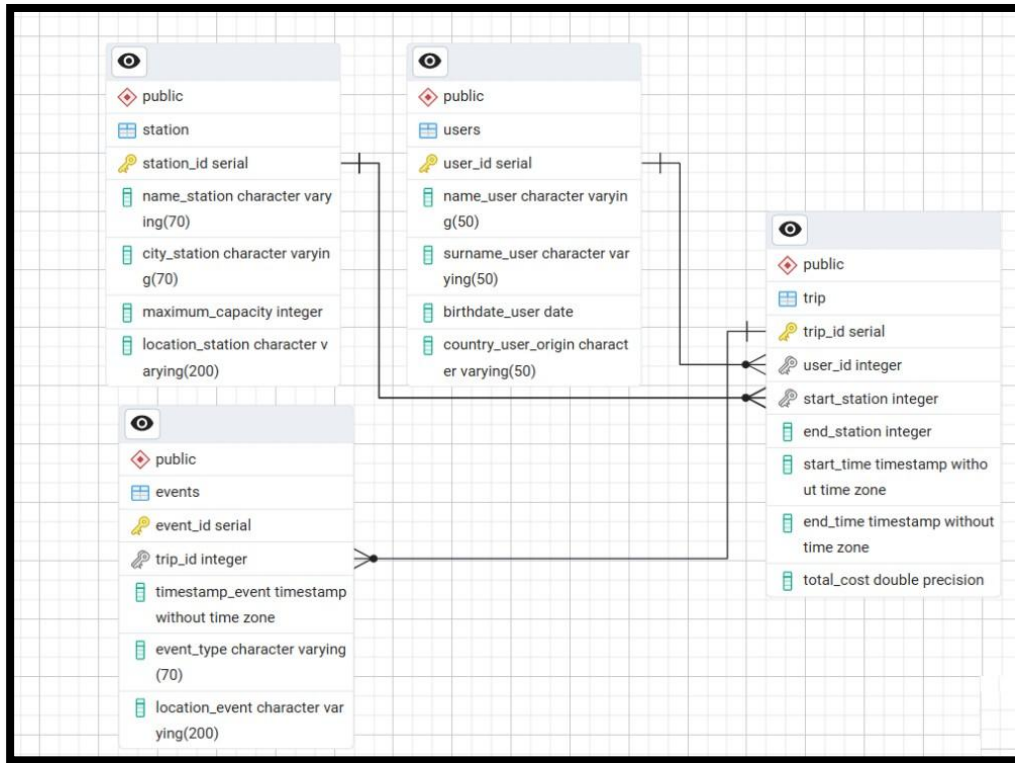


Figure 3-The Relational Data Base Scheme and the relation between different Identities.

Firstly, we create a database called “MobilityPlatform_DB” by using PostgreSQL application.

```

1  -- Create database
2  create database MobilityPlatform_DB;
3
4

```

Figure 4-Creating DataBase in RDBS

Table 1-Database Creation Script (RDBMS)

Create DataBase	
<code>create database</code>	MobilityPlatform_DB;

Then, we create tables of Users, Station, Trip and Events.



The screenshot shows a PostgreSQL SQL editor window with the title "code-mobility.sql* X". The connection is "postgres/postgres@PostgreSQL 18". The editor contains SQL scripts for creating four tables: Users, Station, Trips, and Events. The scripts are as follows:

```
4  --Create Table Users
5  create table users(
6  user_id serial primary key,
7  name_user varchar(50) not null,
8  surname_user varchar(50) not null,
9  birthdate_user date,
10 country_user_origin varchar(50)
11 );
12
13 --Create Table Station
14 create table station(
15 station_id serial primary key,
16 name_station varchar(70) not null,
17 city_station varchar(70) not null,
18 maximum_capacity int,
19 location_station varchar(200)
20 );
21
22 --Create Table Trips
23 create table trip(
24 trip_id serial primary key,
25 user_id int not null,
26 start_station int not null,
27 end_station int not null,
28 start_time timestamp not null,
29 end_time timestamp,
30 total_cost float,
31
32 foreign key (user_id) references users(user_id),
33 foreign key (start_station) references station(station_id),
34 foreign key (end_station) references station(station_id)
35 );
36
37 --Create Table Events
38 create table event(
39 event_id serial primary key,
40 trip_id int not null,
41 timestamp_event timestamp not null,
42 event_type varchar(70) not null,
43 location_event varchar(200),
44 foreign key (trip_id) references trip(trip_id)
45 );
```

Figure 5-The scripts of Table creation in RDBS

Table 2-The scripts of Table creation in RDBS

Create Table of Users
<pre>-- Create Table Users create table users (user_id serial primary key, name_user varchar(255) not null, surname_user varchar(255) not null, birthdate_user date, country_user_origin varchar(255));</pre>
Create Table of Station
<pre>-- Create Table Station create table station(station_id serial primary key, name_station varchar(70) not null, city_station varchar(70) not null, maximum_capacity int, location_station varchar(200));</pre>
Create Table of Trips
<pre>--Create Table Trips create table trip(trip_id serial primary key, user_id int not null, start_station int not null, end_station int not null, start_time timestamp not null, end_time timestamp, total_cost float, foreign key (user_id) references users(user_id), foreign key (start_station) references station(station_id), foreign key (end_station) references station(station_id));</pre>
Create Table of Events
<pre>--Create Table Events create table events (event_id serial primary key, trip_id int not null, timestamp_event timestamp not null, event_type varchar(70) not null, location_event varchar(200), foreign key (trip_id) references trip(trip_id));</pre>

1.2.1.2 Design a document schema in a document-oriented NoSQL DBMS.

Preview of documents	
<pre>_id: ObjectId('69f0bfc14db2abe2ee3f9bf3') user_id: 1 name: "ali" surname: "Ahmadi" birthdate: "10 Jun 1899" country: "Iran"</pre>	<pre>_id: ObjectId('69f0bfc14db2abe2ee3f9bf4') user_id: 2 name: "shima" surname: "shojaei" birthdate: "20 Jun 1990" country: "Iran"</pre>
Preview of documents	
<pre>_id: ObjectId('69f0c7c916e986c11aec5e38') event_id: 1 trip_id: 1 timestamp_event: Code('function(e){if(void... event_type: "error" value_event: 1</pre>	<pre>_id: ObjectId('69f0c7c916e986c11aec5e39') event_id: 2 trip_id: 2 timestamp_event: Code('function(e){if(void... event_type: "error, battery" value_event: 2</pre>
Preview of documents	
<pre>_id: ObjectId('69f0b69f184d0a0b1b4f45f2') id: 1 name_station: "Duomo" city_station: "Milan" maximum_capacity: 1000 location_station: "Piazza del Duomo, 20122 Milano, Italy"</pre>	<pre>_id: ObjectId('69f0b69f184d0a0b1b4f45f3') id: 2 name_station: "Zara" city_station: "Milan" maximum_capacity: 500 location_station: "Viale Zara, 20159 Milano, Italy"</pre>
Preview of documents	
<pre>_id: ObjectId('69f0bba248dd09be97990ed2') trip_id: 1 user_id: 1 start_station: 1 end_station: 3 start_time: Code('function(e){if(void 0===... end_time: Code('function(e){if(void 0===e)... total_cost: 5</pre>	<pre>_id: ObjectId('69f0bba248dd09be97990ed3') trip_id: 2 user_id: 1 start_station: 2 end_station: 1 start_time: Code('function(e){if(void 0===... end_time: Code('function(e){if(void 0===e)... total_cost: 1</pre>

Figure 6-The Schema of the Document-Based Database

Table 3-The scripts of the creation the Tables in document -based DB

Create Database
<code>use("mobility_platform")</code>
Create Table of Users, Stations, Trips and Events
<pre> db.createCollection("users") db.createCollection("stations") db.createCollection("trips") db.createCollection("events") </pre>
Insert data Multiple documents to Users Table
<pre> db.users.insertMany([{'user_id': 1, 'name':'ali', 'surname':'Ahmadi', 'birthdate': '10 Jun 1899', 'country': 'Iran'}, {'user_id': 2, 'name':'shima', 'surname':'shojaei', 'birthdate': '20 Jun 1990', 'country': 'Iran'}, {'user_id': 3, 'name':'Shadi', 'surname':'Shakibaei', 'birthdate': '10 July 2005', 'country': 'Iran'}]) </pre>
Insert data Multiple documents to Stations Table
<pre> db.stations.insertMany([{'id': 1, 'name_station':'Duomo', 'city_station':'Milan', 'maximum_capacity': 1000, 'location_station': 'Piazza del Duomo, 20122 Milano, Italy'}, {'id': 2, 'name_station':'Zara', 'city_station':'Milan', 'maximum_capacity': 500, 'location_station': 'Viale Zara, 20159 Milano, Italy'}, {'id': 3, 'name_station':'Isola', 'city_station':'Milan', 'maximum_capacity': 2000, 'location_station': 'Via Volturno, 20124 Milano, Italy'}]) </pre>
Insert data Multiple documents to Trips Table
<pre> db.trips.insertMany ([{'trip_id': 1, 'user_id':1, 'start_station': 1, 'end_station': 3,'start_time':ISODate, 'end_time': ISODate, 'total_cost':5}, {'trip_id': 2, 'user_id':1, 'start_station': 2, 'end_station': 1,'start_time':ISODate, 'end_time': ISODate, 'total_cost':1}, </pre>

```
{'trip_id': 3, 'user_id':2, 'start_station': 2, 'end_station':3,'start_time':ISODate, 'end_time':ISODate, 'total_cost':1}
```

```
])
```

```
db.trips.insertMany ([{'trip_id': 1, 'user_id':1, 'start_station': 1,
    'end_station': 3,'start_time':ISODate, 'end_time': ISODate, 'total_cost':5},
    {'trip_id': 2, 'user_id':1, 'start_station': 2, 'end_station': 1,'start_time':ISODate, 'end_time': ISODate, 'total_cost':1},
    {'trip_id': 3, 'user_id':2, 'start_station': 2, 'end_station':3,'start_time':ISODate, 'end_time': ISODate, 'total_cost':1}
])
```

Insert data Multiple documents to Events Table

```
db.events.insertMany ([{'event_id': 1, 'trip_id':1, 'timestamp_event':
ISODate, 'event_type': 'error', 'value_event':1},
    {'event_id': 2, 'trip_id':2, 'timestamp_event': ISODate, 'event_type':
    'error, battery', 'value_event':2},
    {'event_id': 3, 'trip_id':2, 'timestamp_event': ISODate,
    'event_type':'no', 'value_event':0}
```

```
])
```

```
db.events.insertMany ([{'event_id': 1, 'trip_id':1, 'timestamp_event': ISODate,
    'event_type': 'error', 'value_event':1}, {'event_id': 2, 'trip_id':2, 'timestamp_event': ISODate,
    'event_type': 'error, battery', 'value_event':2}, {'event_id': 3, 'trip_id':2, 'timestamp_event': ISODate,
    'event_type':'no', 'value_event':0}
])
```

There are some points in Mongo. First of all, we do not have primary and **foreign keys** because it personally gives one automatic number as ID to each field. Besides, it can identify the connection between tables.

1.2.2 Query Implementation

In this part we have to create the random data as inputs in both databases. Then, after implementing the 4 queries which are defined, the scalability has to be compared.

1.2.2.1 Generating simulated data to populate schemas

In your comparison, generate simulated data to **populate** your **schema**, and evaluate the **scalability** of your implementations according to the following parameters:

Number of users:	1k, 10k, 50k
Number of trips:	10k, 50k, 100k
Number of events per trip:	0, 2, 5, 10

1.2.2.1.1 Implement Queries in an RDBMS

In order to evaluate all combinations of values for the three parameters (a total of 36 combinations) and to generate the data randomly, I implemented the process in Python. It is shown in the following table.

Table 4-Python Script for Random Data Generation in RDBMS

Implemented a Python-based data generation script for the process of randomizing in RDBMS

```
import psycopg2
from psycopg2.extras import execute_values
from faker import Faker
import time
import random
from datetime import timedelta

fake=Faker()
fake_it = Faker('it_IT')
stations = []

#1.Connection
conn = psycopg2.connect(
    dbname="mobilityplatform_db",
    user="postgres",
    password="1161369fatemeh",
    host="localhost",
    port="5432"
)
cur = conn.cursor()

# 2.Inputs
user_counts = [1000, 10000, 50000] # users
trip_counts = [10000, 50000, 100000] #trips
events_options = [0, 2, 5, 10]

num_station=120

#3. Clear (Function)
def clear_information(conn):
    with conn.cursor() as cur:
        cur.execute(" TRUNCATE TABLE users, trip, events RESTART IDENTITY CASCADE;")
        conn.commit()

# 4 users-insert (Function)
def insert_users(conn, n_users):
    users = []
    for i in range(n_users):
        users.append((
            fake.first_name(),
            fake.last_name(),
            fake.date_of_birth(minimum_age=18, maximum_age=70),
            fake.country()
        ))
    query = "INSERT INTO users (name_user, surname_user, birthdate_user, country_user_origin) VALUES %s"
```

```

with conn.cursor() as cur:
    execute_values(cur, query, users)
    conn.commit()

# 5.station-insert (Function)
def insert_stations(conn, n_stations):
    stations = []
    for i in range(n_stations):
        #Creating the real name of Italian Cities
        city = fake_it.city()
        station_name = f"station_{city}_{i+1}"

        stations.append((
            station_name,
            city,
            fake.random_int(5, 1000),    # Maximum_capacity
            f"POINT({fake_it.longitude()} {fake_it.latitude()})" # location_station
        ))
        query1= " INSERT INTO station (name_station, city_station, maximum_capacity, location_station)
VALUES %s"

    with conn.cursor() as cur:
        execute_values(cur, query1, stations)
        conn.commit()
    print(f"Successfully inserted {n_stations} stations.")

# 6.Trip-insert (Function)
def insert_trips(conn, n_trips):
    with conn.cursor() as cur:
        cur.execute("SELECT user_id FROM users")
        user_ids = [row[0] for row in cur.fetchall()]

        cur.execute("SELECT station_id FROM station")
        station_ids = [row[0] for row in cur.fetchall()]
        trips = []
        for i in range(n_trips):
            # The random selection of user, start and end station
            user_id = random.choice(user_ids)
            start_station = random.choice(station_ids)
            end_station = random.choice(station_ids)
            start_time = fake.date_time_between(start_date='-30d', end_date='now') # a range for one month among 30
days ago and now.
            end_time = start_time + timedelta(hours=random.randint(1, 5)) #Start Time + Timedelta = End
Time #10+3hours=1pm

```

```

        total_cost = round(random.uniform(5.0, 50.0), 2) # total_cost:Minimum price(5)+Maximum price(50),
        Rounds to two decimal places for cents(2)
        trips.append((
            user_id,
            start_station,
            end_station,
            start_time,
            end_time,
            total_cost
        ))

        #print(f"Successfully inserted {len(trips)} of {n_trips} into the new trips.")
        query2 = "INSERT INTO trip (user_id, start_station, end_station, start_time, end_time, total_cost)
VALUES %s"
        execute_values(cur, query2, trips)
        conn.commit()

# 7.Event-insert
def insert_events(conn, events):
    if events == 0:
        print ("We don't have any event")
        return
    with conn.cursor() as cur:
        cur.execute("SELECT trip_id, start_time, end_time FROM trip") # recieve trip information
        trips = cur.fetchall()

        all_events = []
        event_types = ['Delays', 'Battery', 'GPS', 'Errors']
        for trip in trips: #for each loop, one of raw which are from trips table
            query3 = "INSERT INTO events (trip_id, timestamp_event, event_type, location_event) VALUES %s"
            execute_values(cur, query3, all_events)
            conn.commit()
        print ("events are submitted successfully.")

# 8.run-Query1
def run_query1(conn):
    start = time.time()
    # "Return all trips with: user information, start and end station names" # Joing users, trip, and station tables-
    "trip information and name of station"
    with conn.cursor() as cur:
        query4= """SELECT t.trip_id, u.user_id, u.name_user, u.surname_user, u.country_user_origin,
            st1.name_station as start_station, st2.name_station AS end_station

            FROM trip as t join users as u on t.user_id=u.user_id
                join station as st1 on t.start_station=st1.station_id
                join station as st2 on t.end_station=st2.station_id
            """

```

```

        cur.execute(query4)
        cur.fetchall()
        return time.time()-start
# 9.run-Query2
def run_query2(conn):
    start = time.time()
    # "Return all users with: number of trips performed, average trip duration" #GROUP BY, user information,
    average.time,
    with conn.cursor() as cur:
        query5="""select u.user_id, u.name_user, u.surname_user, COUNT(t.trip_id) as total_trips,
            AVG((t.end_time-t.start_time)) as avg_tripduration
            from users as u left join trip as t on u.user_id=t.user_id
            GROUP BY u.user_id, u.name_user, u.surname_user;
            """
        cur.execute(query5)
        cur.fetchall()
        return time.time() - start
# 10.run-Query3
def run_query3(conn):
    start = time.time()
    # "Return all stations with: number of trips starting there, number of trips ending there;" # number of stations
    with conn.cursor() as cur:
        query6=""" select st.name_station,
            (select COUNT(*) from trip t where t.start_station = st.station_id) as trips_started,
            (select COUNT(*) from trip t where t.end_station = st.station_id) as trips_ended
            from station st;
            """
        cur.execute(query6)
        cur.fetchall()
        return time.time() - start

# 11.run-Query4
def run_query4(conn):
    start = time.time()
    # "Return all trips that contain at least one event with type ERROR;"m #filter Error
    with conn.cursor() as cur:
        query7="""
            SELECT DISTINCT t.trip_id, t.user_id, t.start_time
            FROM trip AS t join events as e on t.trip_id = e.trip_id
            where e.event_type = 'ERROR';
            """
        cur.execute(query7)
        cur.fetchall()
        return time.time() -start
number_insert=1

```

```

results=[]
# main function-part
insert_stations(conn, num_station)
for n_users in user_counts:
    for n_trips in trip_counts:
        for events in events_options:

            print(f"Execution Round #{number_insert}")
            print(f" Users={n_users}, Trips={n_trips}, Events={events}")

            number_insert +=1
            clear_information(conn)
            insert_users(conn,n_users)
            #insert_stations(conn, num_station)
            insert_trips(conn,n_trips)
            insert_events(conn,events)
            time_q1 = run_query1(conn)
            time_q2 = run_query2(conn)
            time_q3 = run_query3(conn)
            time_q4 = run_query4(conn)
            results.append ([n_users, num_station, n_trips, events, time_q1, time_q2, time_q3, time_q4])
            print(f"Results: Q1={time_q1:.4f}s, Q2={time_q2:.4f}s, Q3={time_q3:.4f}s,
Q4={time_q4:.4f}s")
            print("Test Case Completed Successfully.\n")
            print("\n-----")
file_name = "performance_results.csv"
header = ["Users", "Stations", "Trips", "Events", "Time_Q1", "Time_Q2", "Time_Q3", "Time_Q4"]

with open(file_name, mode='w', newline="") as f:
    writer = csv.writer(f)
    writer.writerow(header)
    writer.writerows(results)
print(f"✔ All results saved successfully in {file_name}!")

```

Table of Users

990	990	Latasha	Hayes	1995-01-20	Malaysia
991	991	Jessica	Moon	1981-10-29	Seychelles
992	992	Jessica	Durham	1964-02-19	Armenia
993	993	Jennifer	Campbell	1957-08-15	Uganda
994	994	Martin	Lee	1973-04-16	Cayman Islands
995	995	Lindsay	White	1986-08-24	Liberia
996	996	Carol	Austin	2006-02-26	Finland
997	997	Brian	Guerrero	1987-02-01	Turks and Caicos Islands
998	998	Scott	Joseph	1985-06-10	Norfolk Island
999	999	Jasmine	Jones	1995-08-11	China
1000	1000	Connor	Ramsey	1993-05-08	Montenegro

Table of Station

Data Output

Messages

Notifications

Showing rows: 1 to 120

Page No:

1

of 1

	station_id [PK] integer	name_station character varying (255)	city_station character varying (70)	maximum_capacity integer	location_station character varying (255)
106	391	station_Cella Dati_106	Cella Dati	493	POINT(-136.348168 54.106285...
107	392	station_Castelleone Di Suasa_107	Castelleone Di Suasa	597	POINT(-37.636251 -59.6782235)
108	393	station_Tavullia_108	Tavullia	326	POINT(120.128679 -80.278179...
109	394	station_Scandolara Ripa D'Oglio_109	Scandolara Ripa D'Oglio	470	POINT(127.846146 -68.619800...
110	395	station_Torrette_110	Torrette	931	POINT(66.305456 49.7424475)
111	396	station_Pievebelvicino_111	Pievebelvicino	569	POINT(12.300102 -84.3450445)
112	397	station_Grassano_112	Grassano	293	POINT(-26.446041 21.625767)
113	398	station_Villa Fontana_113	Villa Fontana	140	POINT(51.457046 -58.9211925)
114	399	station_Montecchio Precalcino_114	Montecchio Precalcino	945	POINT(119.258125 85.0319235)
115	400	station_Taranto_115	Taranto	128	POINT(-80.434240 -42.122653)
116	401	station_Baggio_116	Baggio	701	POINT(63.566229 -20.291357)
117	402	station_Argusto_117	Argusto	664	POINT(14.237362 57.7928645)
118	403	station_Moerna_118	Moerna	142	POINT(51.716738 -42.8322325)
119	404	station_Grotteria_119	Grotteria	954	POINT(34.473087 43.231186)
120	405	station_San Costantino Di Briatico_120	San Costantino Di Briatico	175	POINT(-65.750414 -77.863386)

Table of Trip

Data Output

Messages

Notifications

Showing rows: 1 to 1000

Page No:

1

of 100

	trip_id [PK] integer	user_id integer	start_station integer	end_station integer	start_time timestamp without time zone	end_time timestamp without time zone	total_cost double precision
986	986	13918	351	288	2026-05-01 20:19:47	2026-05-02 00:19:47	44.
987	987	19032	317	383	2026-04-09 14:44:58	2026-04-09 15:44:58	12.
988	988	2377	384	393	2026-04-26 11:51:00	2026-04-26 14:51:00	11.
989	989	46052	385	402	2026-04-21 03:01:14	2026-04-21 08:01:14	46.
990	990	35059	348	383	2026-04-14 18:00:31	2026-04-14 21:00:31	48.
991	991	48983	339	295	2026-04-30 19:08:32	2026-04-30 22:08:32	30.
992	992	1783	334	355	2026-04-17 06:42:58	2026-04-17 09:42:58	10.
993	993	9710	310	365	2026-05-01 22:03:04	2026-05-02 01:03:04	20.
994	994	45321	361	398	2026-04-11 14:00:18	2026-04-11 18:00:18	24.
995	995	14666	370	376	2026-04-19 22:47:19	2026-04-20 01:47:19	49.
996	996	25898	382	364	2026-04-09 03:59:21	2026-04-09 07:59:21	20.
997	997	40639	290	363	2026-04-11 17:37:57	2026-04-11 21:37:57	5.
998	998	2973	362	392	2026-04-25 07:53:26	2026-04-25 12:53:26	40.
999	999	21247	381	289	2026-05-01 23:47:21	2026-05-02 00:47:21	39.
1000	1000	22763	287	323	2026-04-20 00:41:10	2026-04-20 03:41:10	13.

Table of Events

count bigint
1

	event_id [PK] integer	trip_id integer	timestamp_event timestamp without time zone	event_type character varying (70)	location_event character varying (255)
988	988	99	2026-04-18 23:20:04	GPS	POINT(-81.1067625 -106.9825...
989	989	99	2026-04-19 00:16:30	Delays	POINT(-76.827281 -78.842164)
990	990	99	2026-04-18 23:43:24	GPS	POINT(-52.2153635 -167.2410...
991	991	100	2026-04-10 01:10:54	GPS	POINT(70.0535575 -77.020349)
992	992	100	2026-04-10 02:51:19	Errors	POINT(-3.782434 6.866258)
993	993	100	2026-04-09 22:55:10	Errors	POINT(-4.805632 -83.800026)
994	994	100	2026-04-10 01:00:16	Errors	POINT(-19.4109145 28.879077)
995	995	100	2026-04-10 01:30:10	Errors	POINT(38.065975 174.182851)
996	996	100	2026-04-10 03:10:23	Delays	POINT(57.3176575 -83.610520)
997	997	100	2026-04-10 03:41:28	GPS	POINT(-20.1335935 -68.504625)
998	998	100	2026-04-10 03:42:42	Delays	POINT(48.0724525 27.386593)
999	999	100	2026-04-09 23:01:16	Errors	POINT(-42.0639275 101.26813...
1000	1000	100	2026-04-10 03:36:01	Battery	POINT(-16.7339705 -69.862941)

Table 5-The times of queries implementation

RDBMS (Relational)									
Num	Query				Users	Trips	Events	Minimum Time	Maximum Time(slower)
	Q1	Q2	Q3	Q4					
1	0.07876	0.02404	0.19502	0.00676	1000	1000	0	Q4	Q3
2	0.02981	0.0097	0.23088	0.00477	1000	10000	2	Q4	Q3
3	0.02876	0.01141	0.22593	0.00623	1000	10000	5	Q2	Q3
4	0.03646	0.01882	0.5441	0.03085	1000	10000	10	Q2	Q3
5	0.21659	0.04473	1.97172	0.00566	1000	50000	0	Q4	Q3
6	0.1829	0.02115	1.72775	0.04384	1000	50000	2	Q2	Q3
7	0.15264	0.04743	1.58288	0.12305	1000	50000	5	Q2	Q3
8	0.18161	0.0312	0.91947	0.12921	1000	50000	10	Q2	Q3
9	0.34689	0.06727	2.78758	0.00508	1000	100000	0	Q4	Q3
10	0.55963	0.12117	3.74944	0.12503	1000	100000	2	Q2	Q3
11	0.38623	0.06018	2.23017	0.12446	1000	100000	5	Q2	Q3
12	0.2901	0.07896	1.88293	0.20631	1000	100000	10	Q2	Q3
13	0.1079	0.08872	0.49067	0.00503	10000	10000	0	Q4	Q2
14	0.10707	0.11055	0.49171	0.00933	10000	10000	2	Q4	Q3
15	0.11544	0.11788	0.50686	0.01997	10000	10000	5	Q4	Q3
16	0.11626	0.10769	0.48464	0.03828	10000	10000	10	Q4	Q3
17	0.46643	0.22414	2.52888	0.00469	10000	50000	0	Q4	Q3
18	0.12588	0.05188	1.92258	0.0433	10000	50000	2	Q4	Q3
19	0.24572	0.14266	2.29748	0.08806	10000	50000	5	Q4	Q3
20	0.14247	0.07346	0.85434	0.09205	10000	50000	10	Q2	Q3
21	0.40267	0.09022	2.04772	0.00262	10000	100000	0	Q4	Q3
22	0.39089	0.11221	2.88924	0.11169	10000	100000	2	Q4	Q3
23	0.34016	0.14367	1.74666	0.09946	10000	100000	5	Q4	Q3
24	0.24203	0.07112	4.09645	0.20994	10000	100000	10	Q2	Q3
25	0.04954	0.16366	0.21834	0.00369	50000	10000	0	Q4	Q3
26	0.11657	0.23554	0.30327	0.00959	50000	10000	2	Q4	Q3
27	0.06219	0.11352	0.40251	0.01004	50000	10000	5	Q4	Q3
28	0.05913	0.13046	0.26818	0.03358	50000	10000	10	Q4	Q3
29	0.17596	0.52771	2.29124	0.00485	50000	50000	0	Q4	Q3
30	0.14698	0.37895	2.06609	0.02601	50000	50000	2	Q4	Q3

31	0.13528	0.18833	2.31689	0.15363	50000	50000	5	Q1	Q3
32	0.15357	0.44796	1.27697	0.19122	50000	50000	10	Q1	Q3
33	0.9302	0.74989	4.47552	0.00378	50000	100000	0	Q4	Q3
34	0.38909	0.3082	4.65692	0.12613	50000	100000	2	Q4	Q3
35	0.28377	0.51088	4.78195	0.17297	50000	100000	5	Q4	Q3
36	0.42714	0.35391	2.47492	0.24613	50000	100000	10	Q4	Q3
Min	0.02981 (Round2)	0.0097 (Round2)	0.19502 (Round1)	0.00262 (Round21)					
Max	0.9302 (Round33)	0.74989 (Round33)	4.78195 (Round35)	0.24613 (Round36)					

➤ Query Performance and Efficiency Analysis:

Based on the experimental results recorded in Table, the execution behavior of the four queries (Q1 to Q4) under scaling scenarios reveals distinct performance characteristic:

✓ **Q4** (Return all trips that contain at least one event with type ERROR):

as the Fastest Query, across almost all **36 simulation rounds**, Q4 consistently achieved the **Minimum Time**, often executing **in less than 0.01 seconds**. This exceptional efficiency is due to its selective filtering nature, which targets a highly restricted subset **of event types** without demanding heavy aggregation or multi-table indexing overhead.

✓ **Q3**: (Return all stations with: number of trips starting there, number of trips ending there;) In contrast, was persistently classified as the Maximum Time (slower) query, **with execution times** escalating up **to 4.78 seconds** in higher **data volumes** (e.g., Round 35). The **performance constraint** is because the system has to run two separate grouping and counting operations on the large Trips dataset—one for starting trips and one for ending trips at each station.

✓ **Q1 and Q2**

- (1. Return all trips with: user information, start and end station names;
2. Return all users with: number of trips performed, average trip duration;)

Scalability of Q1 and Q2: Both Q1 (multi-table join) and Q2 (user-trip metrics calculation) exhibited stable, sub-second execution times in the majority of rounds. While their **execution times** grew logically as the number of **users scaled** to 50k and trips to 100k, the Relational Database Management System (RDBMS) successfully optimized these **joins** via **foreign-key structural indexing**.

1.2.2.1.2 A document-oriented NoSQL DBMS

A document-based database like MongoDB, we do not rely on traditional joins (or we try to avoid them as much as possible). In **MongoDB**, data is typically **modeled** using **nested structures** or **processed** through the **Aggregation Pipeline**.

Table 6- Python Script for Random Data Generation in document-based DB

Implemented a Python-based data generation script for the process of randomizing in Mongo

```
import random
from pymongo import MongoClient
from datetime import timedelta
from faker import Faker
import time
import csv

fake=Faker()
fake = Faker('it_IT')
fake_it = Faker('it_IT')

#1.Connection
client = MongoClient("mongodb://localhost:27017/")
db = client["mobility_platform"]

users_collection = db["users"]
trips_collection = db["trips"]
stations_collection = db["stations"]
events_collection = db["events"]

# 2.Inputs
user_counts = [1000, 10000, 50000]
trip_counts = [10000, 50000, 100000]
events_options = [0, 2, 5, 10]
num_stations = 120

results1=[]

#3. Clear (Function)
def clear_information(db):
    db.users.drop()
    db.trips.drop()
    db.events.drop()

# 4 users-insert (Function)
def insert_users(db, n_users):
    users_collection = db["users"]
    user_doc = []
    for i in range(n_users):

        new_user= {
            "user_id": i+1,
            "name_user": fake.first_name(),
```

```

        "surname_user": fake.last_name(),
        "birthdate": str(fake.date_of_birth(minimum_age=18, maximum_age=70)),
        "country_user_origin": fake.country()
    }
    #adding to list
    user_doc.append(new_user)
    #add into mongo
    if user_doc:
        users_collection.insert_many(user_doc)
    # print(f" Successfully inserted {n_users} users.")

# 5. station-insert (Function)
def insert_stations(db, n_stations):
    stations_collection = db["stations"]
    stations = []
    for i in range(n_stations):
        city = fake_it.city()
        station_name = f"station_{city}_{i+1}"
        station_doc = {
            "station_id": i+1,
            "name_station": station_name,
            "city_station": city,
            "maximum_capacity": fake.random_int(5, 1000),
            "location_station": {
                "type": "Point",
                "coordinates": [float(fake_it.longitude()), float(fake_it.latitude())]
            }
        }
        stations.append(station_doc)
    if stations:
        stations_collection.insert_many(stations)
    # print(f"Successfully inserted {n_stations} stations into MongoDB.")

# 6. Trip-insert (Function)
def insert_trips(db, n_trips):
    trips_collection = db["trips"]
    user_ids = db["users"].distinct("user_id")
    station_ids = db["stations"].distinct("station_id")
    trips = []

    for i in range(n_trips):
        user_id = random.choice(user_ids)
        start_station = random.choice(station_ids)
        end_station = random.choice(station_ids)
        start_time = fake.date_time_between(start_date='-30d', end_date='now')

```

```

end_time = start_time + timedelta(hours=random.randint(1, 5))
total_cost = round(random.uniform(5.0, 50.0), 2) #cost

trip_doc = {
    "user_id": user_id,
    "start_station_id": start_station,
    "end_station_id": end_station,
    "start_time": start_time,
    "end_time": end_time,
    "total_cost": total_cost
}
trips.append(trip_doc)

if trips:
    trips_collection.insert_many(trips)
    print(f'Successfully inserted {n_trips} trips into MongoDB.')

# 7.Event-insert
def insert_events(db, events_per_trip):
    if events_per_trip == 0:
        print("We don't have any event")
        return
    events_collection = db["events"]
    trips = list(db["trips"].find({}, {"_id": 1, "start_time": 1, "end_time": 1}))
    event_types = ['Delays', 'Battery', 'GPS', 'Errors']
    all_events = []
    for trip in trips:
        for _ in range(events_per_trip):
            timestamp = fake.date_time_between(
                start_date=trip["start_time"],
                end_date=trip["end_time"]
            )

            event_doc = {
                "trip_id": trip["_id"],
                "timestamp_event": timestamp,
                "event_type": random.choice(event_types),
                "location_event": {
                    "type": "Point",
                    "coordinates": [float(fake.longitude()), float(fake.latitude())]
                }
            }
            all_events.append(event_doc)

    if all_events:

```

```

events_collection.insert_many(all_events)
print(f" Events submitted successfully for {len(all_events)} records.")

# 8.run-Query1
def run_query1(db):
    start = time.time()
    #"Return all trips with: user information, start and end station names" # Joing users, trip, and station tables-
    "trip information and name of station"
    query1= [
        {
            # 1. Join with users collection
            "$lookup":{
                "from": "users",
                "localField": "user_id",
                "foreignField": "user_id",
                "as": "user_info"
            }
        },
        # convert user array into object
        {"$unwind": "$user_info"},
        # 2. Join with stations collection
        {
            "$lookup": {
                "from": "stations",
                "localField": "start_station_id",
                "foreignField": "station_id",
                "as": "start_st_info"
            }
        },
        {"$unwind": "$start_st_info"},
        # 3. Join with stations collection for the end-station
        {
            "$lookup":{
                "from": "stations",
                "localField": "end_station_id",
                "foreignField": "station_id",
                "as": "end_st_info"
            }
        },
        {"$unwind": "$end_st_info"},
        # 4. Now, selection the main fields
        {
            "$project": {
                "trip_id": 1,
                "user_id": "$user_info.user_id",

```

```

        "name_user": "$user_info.name_user",
        "surname_user": "$user_info.surname_user",
        "country_user_origin": "$user_info.country_user_origin",
        "start_station": "$start_st_info.name_station",
        "end_station": "$end_st_info.name_station",
        "_id": 0
    }
}
]
results = list(db["trips"].aggregate(query1))
return time.time() - start

# 9.run-Query2
def run_query2(db):
    start = time.time()
    #"Return all users with: number of trips performed, average trip duration" #GROUP BY, user information,
    average.time.
    query2= [
        # 1. Join with trips collection (LEFT JOIN)
        {
            "$lookup": {
                "from": "trips",
                "localField": "user_id",
                "foreignField": "user_id",
                "as": "user_trips"
            }
        },
        {
            "$unwind": {
                "path": "$user_trips",
                "preserveNullAndEmptyArrays": True
            }
        },
        # 3. time
        {
            "$project": {
                "user_id": 1,
                "name_user": 1,
                "surname_user": 1,
                "duration": {
                    "$subtract": ["$user_trips.end_time", "$user_trips.start_time"]
                },
                "has_trip": { "$cond": [{ "$ifNull": ["$user_trips.trip_id", False] }, 1, 0] }
            }
        },
    ],

```

```

# 4. (GROUP BY based on group and duration
{
  "$group": {
    "_id": {
      "user_id": "$user_id",
      "name_user": "$name_user",
      "surname_user": "$surname_user"
    },
    "total_trips": { "$sum": "$has_trip" }, # Trip Numbers
    "avg_tripduration": { "$avg": "$duration" } # Time Average
  }
},
# 5. User
{
  "$project": {
    "user_id": "$_id.user_id",
    "name_user": "$_id.name_user",
    "surname_user": "$_id.surname_user",
    "total_trips": 1,
    "avg_tripduration": 1,
    "_id": 0
  }
}
]
results = list(db["users"].aggregate(query2))
return time.time() - start
# 10.run-Query3
def run_query3(db):
    start = time.time()
    query3 = [
        # 1. number of trips which are started
        {
            "$lookup": {
                "from": "trips",
                "localField": "station_id",
                "foreignField": "start_station_id",
                "as": "started_docs"
            }
        },
        # 2. number of trips which are finished in that station
        {
            "$lookup": {
                "from": "trips",
                "localField": "station_id",
                "foreignField": "end_station_id",

```

```

        "as": "ended_docs"
    }
},
# 3. convert list to number
{
    "$project": {
        "name_station": 1,
        "trips_started": { "$size": "$started_docs" },
        "trips_ended": { "$size": "$ended_docs" },
        "_id": 0
    }
}
]
results = list(db["stations"].aggregate(query3))
return time.time() - start

# 11.run-Query4
def run_query4(db):
    start = time.time()
    #Return all trips that contain at least one event with type ERROR;"m #filter Error
    query4 = [
        # 1. Join events Collection
        {
            "$lookup": {
                "from": "events",
                "localField": "trip_id",
                "foreignField": "trip_id",
                "as": "trip_events"
            }
        },
        # 2. Filtering the trips which have at least one ERROR
        {
            "$match": {
                "trip_events.event_type": "ERROR"
            }
        },
        # 3. Remove repetitive items (DISTINCT)
        # each trip(trip_id) presents once
        {
            "$group": {
                "_id": "$trip_id",
                "user_id": { "$first": "$user_id" },
                "start_time": { "$first": "$start_time" }
            }
        },
    ],

```



```

# 4.    {
    "$project": {
        "trip_id": "$_id",
        "user_id": 1,
        "start_time": 1,
        "_id": 0
    }
}
]
results = list(db["trips"].aggregate(query4))
return time.time() - start

# main function-part
insert_num=1
insert_stations(db, num_stations)
# clear_information(db)
for n_users in user_counts:
    for n_trips in trip_counts:
        for events in events_options:
            print(f"Execution Round #{insert_num}")
            # print(f" Users={n_users}, Trips={n_trips}, Events={events}")
            print("\n-----")
            clear_information(db)
            insert_users (db, n_users)
            insert_trips (db, n_trips)
            insert_events (db, events)
            insert_num += 1

            db.users.create_index("user_id")
            db.trips.create_index("user_id")
            db.trips.create_index("start_station_id")
            db.trips.create_index("end_station_id")
            db.events.create_index("trip_id")

            time_q1 = run_query1(db)
            time_q2 = run_query2(db)
            time_q3 = run_query3(db)
            time_q4 = run_query4(db)

            results1.append ([n_users, num_stations, n_trips, events, time_q1, time_q2, time_q3, time_q4])
            print(f"Results: Q1={time_q1:.4f}s, Q2={time_q2:.4f}s, Q3={time_q3:.4f}s, Q4={time_q4:.4f}s")
            print("Test Case Completed Successfully.\n")

            file_name = r"C:\Users\Asus\Desktop\performance_results.csv"
            header = ["Users", "Stations", "Trips", "Events", "Time_Q1", "Time_Q2", "Time_Q3", "Time_Q4"]

```

```
with open(file_name, mode='w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(header)
    writer.writerows(results1)
```

```
-----
Successfully inserted 100000 trips into MongoDB.
We don't have any event
Results: Q1=43.1022s, ,Q2=7.9036s,Q3=0.5514s, Q4=1.0838s
Test Case Completed Successfully.
```

Execution Round #:34

```
-----
Successfully inserted 100000 trips into MongoDB.
Events submitted successfully for 200000 records.
Results: Q1=32.7612s, ,Q2=4.8869s,Q3=0.3737s, Q4=0.7275s
Test Case Completed Successfully.
```

Execution Round #:35

```
-----
Successfully inserted 100000 trips into MongoDB.
Events submitted successfully for 500000 records.
Results: Q1=36.1159s, ,Q2=8.0635s,Q3=0.8973s, Q4=1.1544s
Test Case Completed Successfully.
```

Execution Round #:36

Collection of Users

events

tri... X

stations

users

+

localhost:27017 > mobility_platform > users

> Open MongoDB shell

Documents 50K Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

+

25 1 - 25 of 50000

```
_id: ObjectId('69ff6c0d19b99e444ed84345')
user_id: 1
name_user: "Luchino"
surname_user: "Smirnoff"
birthdate: "1974-02-26"
country_user_origin: "Polonia"
```

```
_id: ObjectId('69ff6c0d19b99e444ed84346')
user_id: 2
name_user: "Gianfranco"
surname_user: "Avogadro"
birthdate: "2008-02-27"
country_user_origin: "Burkina Faso"
```

Collection of Stations

localhost:27017 > mobility_platform > stations

Open MongoDB shell

Documents120

Aggregations

Schema

Indexes1

Validation

🕒

Type a query: { field: 'value' } or [Generate query](#) ⚡

Explain

Reset

Find

Options ▶

+ ▼

25 ▼

1 – 25 of 120

<

>

▼

☰

_id: ObjectId('69ff668b19b99e444e44d49d')

station_id: 1

name_station: "station_Passo Del Tonale_1"

city_station: "Passo Del Tonale"

maximum_capacity: 599

location_station: Object

_id: ObjectId('69ff668b19b99e444e44d49e')

station_id: 2

name_station: "station_Masera' Di Padova_2"

city_station: "Masera' Di Padova"

maximum_capacity: 173

location_station: Object

Collection of Trips

localhost:27017 > mobility_platform > trips

Open MongoDB shell

Documents100K

Aggregations

Schema

Indexes4

Validation

🕒

Type a query: { field: 'value' } or [Generate query](#) ⚡

Explain

Reset

Find

Options ▶

+ ▼

25 ▼

1 – 25 of 100000

<

>

▼

☰

_id: ObjectId('69ff6c1019b99e444ed90695')

user_id: 3964

start_station_id: 120

end_station_id: 50

start_time: 2026-04-17T14:29:22.000+00:00

end_time: 2026-04-17T16:29:22.000+00:00

total_cost: 46.19

_id: ObjectId('69ff6c1019b99e444ed90696')

user_id: 13434

start_station_id: 17

end_station_id: 29

start_time: 2026-05-01T22:21:27.000+00:00

end_time: 2026-05-02T02:21:27.000+00:00

total_cost: 45.26

Table of Events

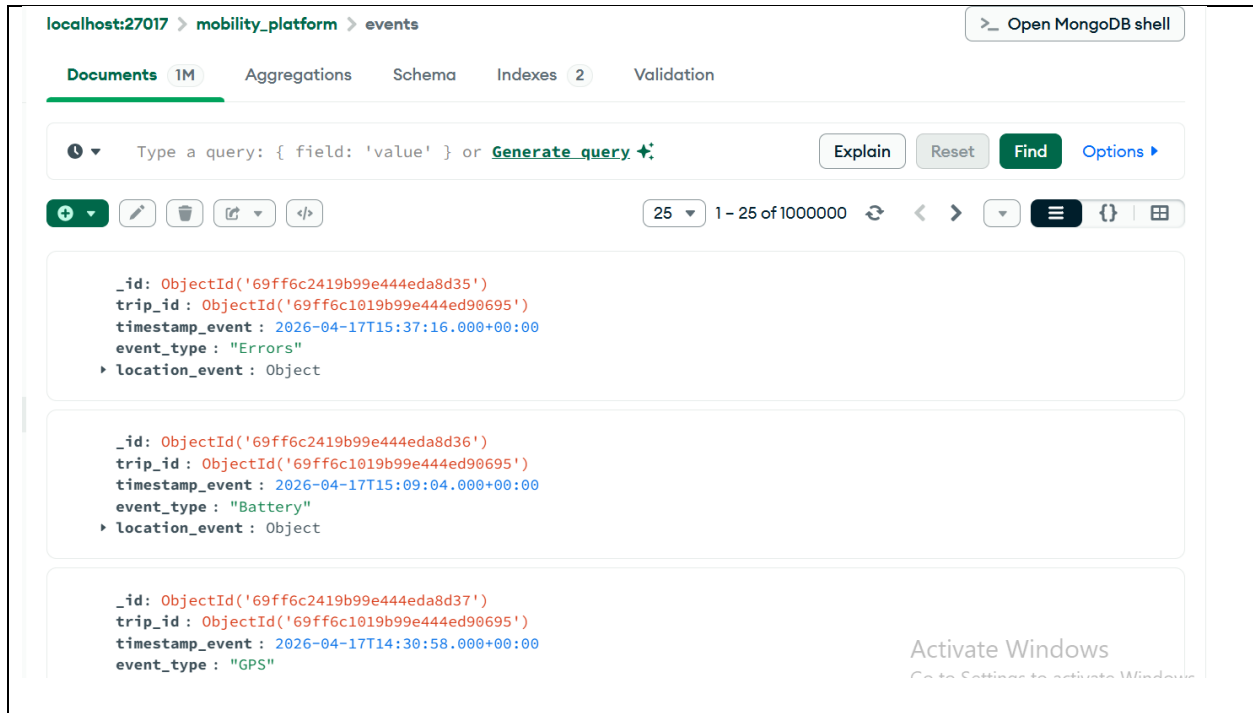


Table 7-The times of queries implementation id Document-based DB

Document-based DB									
Num	Query				Users	Trips	Events	Minimum Time	Maximum Time(slower)
	Q1	Q2	Q3	Q4					
1	9.12313	0.64243	0.25333	0.21106	1000	1000	0	Q4	Q1
2	8.87129	0.47939	0.22893	0.31278	1000	10000	2	Q3	Q1
3	9.00546	0.45124	0.1967	0.32177	1000	10000	5	Q3	Q1
4	8.93797	0.47313	0.16387	0.38553	1000	10000	10	Q3	Q1
5	47.14989	0.99889	0.63598	0.97953	1000	50000	0	Q3	Q1
6	46.74068	1.0946	0.60314	1.13988	1000	50000	2	Q3	Q1
7	45.95114	0.90792	0.58074	1.08026	1000	50000	5	Q3	Q1
8	45.30385	1.10554	0.57964	0.94305	1000	50000	10	Q3	Q1
9	45.14313	0.7918	0.72922	1.09046	1000	100000	0	Q3	Q1
10	45.09594	0.79168	0.6551	1.07259	1000	100000	2	Q3	Q1
11	45.514	0.84575	0.65871	1.0608	1000	100000	5	Q3	Q1
12	48.56093	1.06861	0.75792	1.5042	1000	100000	10	Q3	Q1
13	6.45778	1.7287	0.18216	0.1341	10000	10000	0	Q4	Q1
14	5.81945	1.46737	0.08199	0.27279	10000	10000	2	Q3	Q1
15	5.01344	1.40975	0.08367	0.11961	10000	10000	5	Q3	Q1
16	4.73276	1.30057	0.09732	0.15491	10000	10000	10	Q3	Q1
17	21.6161	1.55019	0.30114	0.45989	10000	50000	0	Q3	Q1
18	22.77542	1.52355	0.27251	0.50911	10000	50000	2	Q3	Q1
19	20.53723	1.50603	0.28905	0.53862	10000	50000	5	Q3	Q1
20	22.78803	1.54576	0.2721	0.51209	10000	50000	10	Q3	Q1
21	43.43289	1.89396	0.53754	0.91065	10000	100000	0	Q3	Q1
22	41.32883	1.98993	0.62186	0.98258	10000	100000	2	Q3	Q1

23	41.82678	1.99859	0.5557	1.1536	10000	100000	5	Q3	Q1
24	41.67921	1.87378	0.5568	1.17238	10000	100000	10	Q3	Q1
25	4.43405	5.85978	0.17096	0.1734	50000	10000	0	Q3	Q2
26	4.47867	5.69473	0.06047	0.11564	50000	10000	2	Q3	Q2
27	4.33581	5.67679	0.06532	0.11499	50000	10000	5	Q3	Q2
28	4.71128	5.713	0.06181	0.11227	50000	10000	10	Q3	Q2
29	21.31591	6.36604	0.25813	0.45673	50000	50000	0	Q3	Q1
30	22.56659	6.50614	0.28219	0.64738	50000	50000	2	Q3	Q1
31	21.99189	7.30368	0.3637	0.56964	50000	50000	5	Q3	Q1
32	21.14969	6.32687	0.25359	0.50886	50000	50000	10	Q3	Q1
33	43.10224	7.90358	0.55143	1.08383	50000	100000	0	Q3	Q1
34	32.76122	4.88687	0.37374	0.7275	50000	100000	2	Q3	Q1
35	36.11593	8.06348	0.89734	1.15437	50000	100000	5	Q3	Q1
36	49.52293	7.99347	0.78008	1.14317	50000	100000	10	Q3	Q1
Min	9.12313 (Round1)	0.64243 (Round1)	0.25333 (Round1)	0.21106 (Round1)					
Max	49.52293 (Round36)	7.99347 (Round136)	0.78008 (Round136)	1.14317 (Round136)					

➤ Query Performance and Efficiency Analysis:

Based on the experimental results recorded in Table, the execution behavior of the four queries (Q1 to Q4) under scaling scenarios reveals distinct performance characteristic:

✓ **Q4** (Return all trips that contain at least one event with type ERROR):

Linear Stability of Q4: Q4 **initially** captured the absolute **minimum time** at **0.21** seconds (Round 1). As the workload scaled, its execution trend remained stable, around 1.14 seconds. Since it only filters error events and doesn't depend on any multi-table relationships, its size grows only based on how many rows the table has.

✓ **Q3:** (Return all stations with: number of trips starting there, number of trips ending there;)

The Superior Scaling **Efficiency** of Q3: **Q3** established itself as the **Minimum Time (Fastest)** query across the majority of the rounds, executing in merely 0.78 seconds even at maximum scale (Round 36). This fast aggregation happens because the Stations table is very small.

✓ **Q2** (2. Return all users with: number of trips performed, average trip duration;)

The **performance of Q2** demonstrates severe sensitivity to **a high number of users**. While maintaining highly efficient **execution times below 2 seconds** when users were bounded at 1k and 10k, it suddenly **slowed down to 7.99** seconds in Round 36 when the user **count reached 50k**. This reveals that computing the **metrics (COUNT and AVERAGE)** per user forces heavy sorting operations that exhaust temporary memory.

✓ Q1

(1. Return all trips with: user information, start and end station names;)

Unlike typical workloads, Q1 consistently recorded the Maximum Time in almost every scenario. **Starting at 9.12 seconds in Round 1**, its execution time scaled exponentially to an absolute peak **of 49.52 seconds in Round 36**. This sharp reduce in time shows that joining multiple tables to get user info and station details forces the system to repeatedly scan the entire Trips table because there are no composite indexes.

1.2.3 Schema Evolution

1.2.3.1 Schema Evolution in RDBS

If we want to handle a future schema modification for the events of type BATTERY which have be included an additional field "battery level" (an integer, with values in [0, 100]), in RDBS, we have to define an additional column for the event table. So, we use "Alter Table" for this table. Default values for the existing records that do not have this field, you must either allow NULL or assign a default value (such as -1 or 0).

The value must be within the range [0, 100]. In SQL, we need to define a "CHECK CONSTRAINT" to prevent inserting data outside this range. Moreover, we have to consider the foreign keys for the "Event Table" such as "Trip_Id" and "Timestamp" with the respect to "event-id".

Table 8-The script of insert additional Event Value

Add new column to the EVENT Table in RDBS
ALTER TABLE events ADD COLUMN battery_level INTEGER;
Adding a constraint to check the value range (0 to 100)
ALTER TABLE events ADD CONSTRAINT check_battery_range CHECK (battery_level >= 0 AND battery_level <= 100);
Add new column to the EVENT Table in RDBS by Python Programming
import psycopg2 from psycopg2.extras import execute_values #1.connection conn = psycopg2.connect(database="mobilityplatform_db", user="postgres", password="1161369fatemeh", host="localhost", port="5432") cur = conn.cursor() #2. Writing query

```

def upgrade_schema():
    alter_query = """
        alter table events
        add column if not exists battery_level integer
        check (battery_level >= 0 and battery_level <= 100);
    """

    try:
        with conn.cursor() as cur:
            cur.execute(alter_query)
            conn.commit()
            print("Schema updated successfully!")
    except:
        print(f"Error")
    upgrade_schema()

```

Add value to the “battery-value” column.

#3. insert value in the battery_value

```

def update_event_type():
    try:
        with conn.cursor() as cur:
            # 1. recieve event-id of events table
            cur.execute("select event_id from events where battery_level is null")
            rows = cur.fetchall()
            for row in rows:
                event_id = row[0]

                # creating random input among 0 to 100
                new_value = random.randint(0, 100)

                # 2. update the records based on (trip_id)
                cur.execute(
                    "update events set battery_level = %s where event_id = %s", #When find event_id, update that
                    (new_value, event_id)
                )

            conn.commit()
            print(f"Successfully updated {len(rows)} records.")
    except Exception as e:
        print(f"Error: {e}")
        conn.rollback()
    update_event_type()

```

1.2.3.2 Schema Evolution in the Document-based

For doing this change in MongoDB is **much more flexible**. There is **no need to modify the entire database by altering table**. In “PostgreSQL”, we face to alter “Event Table” to define new column “Battery_level”, then we have to add values with **the respect to the foreign key**(trip_id). We can manage the all structure in the one area with “update_many”.

Table 9-Comparison between RDBMS and MongoDB regarding to adding a new column

Feature	Relational Database (SQL-PostgreSQL)	Document-Oriented Database (MongoDB)
Ease of Change	Requires alter table (mainly database) (changing scheme), then adding data → which needs more time → this may involve system downtime .	Very simple ; new documents just include the new field . But, in Mongo, it does not need to change the scheme (Schema-less). Not need to change db to add the new field and the old documents remain without change.
Dependency	Primary-key should be considered	Just search the related field in the collections , then entering information in the fields .
		We enter data directly to mongo, if it be in the db, the data is added. In contrast, the field will be created.
Flexibility	Less	More
Time-consuming	More	Less

Table 10-Python Script for Searching and Assigning Battery Values (0–100)

Search and Add value to the “battery-value” column in the value range (0 to 100).

```
import random
from pymongo import MongoClient
#1.connection
client = MongoClient("mongodb://localhost:27017/")
db = client["mobility_platform"]

#2. Writing query
def add_battery_value():
    # 1.finding the documents/collections which do not have field"battery_value
    query = {"battery_value": {"$exists": False}}

    #2.get id of enets table which it is in the query, means not False
    events_to_fix = db.events.find(query, {"_id": 1})

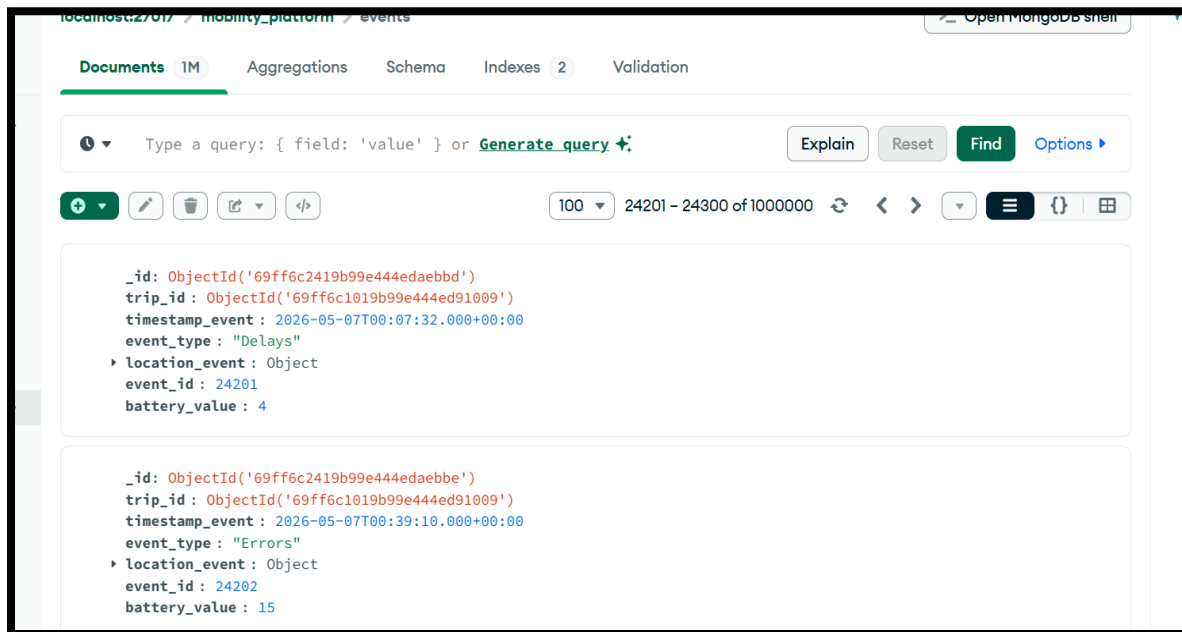
    updates = []
    count = 0
    for event in events_to_fix:
        #3.Creating the random value for input in the battery_value field.
        new_value = random.randint(0, 100)
        db.events.update_one({"_id": event["_id"]}, {"$set": {"battery_value": new_value}})
        count += 1
    if count % 1000 == 0:
        print(f"so far{count} is updated..")

add_battery_value()
```

```

1 import random
2 from pymongo import MongoClient
3
4
5 #1.connection
6 client = MongoClient("mongodb://localhost:27017/")
7 db = client["mobility_platform"]
8
9 #2. Writing query
10 def add_battery_value():
11     # 1.finding the documents/collections which do not have field"battery_value
12     query = {"battery_value": {"$exists": False}}
13
14     #2.get id of enets table which it is in the query, means not False
15     events_to_fix = db.events.find(query, {"_id": 1})
16
17     updates = []
18     count = 0
19     for event in events_to_fix:
20         #3.Greating the random value for input in the battery_value field.
21         new_value = random.randint(0, 100)
22         db.events.update_one({"_id": event["_id"]}, {"$set": {"battery_value": new_value}})
23         count += 1
24     if count % 1000 == 0:
25         print(f"so far{count} is updated..")
26
27 add_battery_value()

```



1.2.4 Spark-based Implementation

First, I installed Apache Spark. Then, I prepared the data in MongoDB and connected to it from Spark through a Spark session. Based on Query 2, we then retrieved the required information. Finally, the first part (installing Spark) was extremely challenging because, due to several system-related issues, I first had to install the standard Java runtime and then set up Hadoop. (1). Initially,

I provide a Spark-based implementation for Query 2 in the document-based implementation of the data management system without timer and considering all various figures.

Table 11-Query 2 Spark Execution in the Document-Based Model (No Timer)

```
import os
import sys
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

#Setting Python for Spark
os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

#Setting Java and Mongo
os.environ['PYSPARK_SUBMIT_ARGS'] = (
    '--conf "spark.driver.extraJavaOptions=--add-opens=java.base/java.lang=ALL-UNNAMED '
    '--add-opens=java.base/javax.security.auth=ALL-UNNAMED" '
    '--packages org.mongodb.spark:mongo-spark-connector_2.12:10.4.0 pyspark-shell'
)

#1. #Creating session of spark with the name of project and Connection with MongoDB and call Name of
project- #find db of mongo\ - #set decision of writing
spark = SparkSession.builder \
    .appName("mobility-platform") \
    .config("spark.mongodb.read.connection.uri", "mongodb://localhost:27017/mobility_platform") \
    .config("spark.mongodb.write.connection.uri", "mongodb://127.0.0.1:27017/mobility_platform") \
    .getOrCreate()

#2.show errors
spark.sparkContext.setLogLevel("ERROR")

#3 read from collections (for inputing information of users and trips)
try:
#df=Data Frame
    df_users = spark.read.format("mongodb") \
        .option("database", "mobility_platform") \
        .option("collection", "users").load()

    df_trips = spark.read.format("mongodb") \
        .option("database", "mobility_platform") \
        .option("collection", "trips").load()

#3.Query 2 "2. Return all users with: number of trips performed, average trip duration;"
```

```

query2_stats = df_users.join(
    df_trips, df_users["user_id"] == df_trips["user_id"], "left"
).groupBy(
    df_users["user_id"], "name_user", "surname_user"
).agg(
    #number of users with the trips which did
    F.count("trip_id").alias("total_trips"),
    # average trip duration
    F.avg(F.unix_timestamp("end_time") - F.unix_timestamp("start_time")).alias("avg_tripduration")
)
#Showing the results
query2_stats.write.format ()
except Exception as e:
    print(f" Happening error during execution: {e}")

```

```

Found org.mongodb#bson-record-codec;5.1.4 in central
:: resolution report :: resolve 1736ms :: artifacts dl 15ms
:: modules in use:
org.mongodb#bson;5.1.4 from central in [default]
org.mongodb#bson-record-codec;5.1.4 from central in [default]
org.mongodb#mongodb-driver-core;5.1.4 from central in [default]
org.mongodb#mongodb-driver-sync;5.1.4 from central in [default]
org.mongodb.spark#mongo-spark-connector_2.12;10.4.0 from central in [default]
-----
|      conf      |      modules      | artifacts |
|      number    | search|dwnld|evicted| number|dwnld| |
|---|---|---|---|---|---|---|
| default      | 5 | 1 | 0 | 0 | 5 | 0 |
|-----|-----|-----|
:: retrieving :: org.apache.spark#spark-submit-parent-1c86e2ff-f88f-4cf8-af51-b20e1acd13a0
confs: [default]
0 artifacts copied, 5 already retrieved (0kB/13ms)
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Happening error during execution: An error occurred while calling o78.save.

```

This picture shows the limited number of results. This is because of limitation of spark.

Secondly, for comparing the efficiency of the Spark-based implementation of query2 with the Mongodb in the given values, we have to use a timer. In Spark, taking time is easy, but we just have to put it in the right place because Spark is lazy and doesn't do anything until you call collect() or show() . The table below shows the results, scripts of the implementation.

Table 12-Query 2 Spark Execution in the Document-Based Model (With Timer)

```

from google.colab import drive
drive.mount('/content/drive')

-----
!unzip -q "/content/drive/My Drive/data-project-s/Test_State.zip" -d /content/extracted_data
-----

!pip install pyspark

-----
import os
import sys
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
import time

```

```

import pandas
import csv

-----

print("$ Running Appachi of Spark...")
spark = SparkSession.builder \
    .appName("mobility-platform-colab") \
    .config("spark.driver.memory", "4g") \
    .master("local[*]") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

# =====
# 4. A loop for running 35 Round
# =====

base_path = "/content/extracted_data/Test_State"
spark_results = []

print("\n Start processing 36 Rounds with for structure...")

for i in range(1, 37):
    state_dir = os.path.join(base_path, f"Test_{i}")

    u_file = os.path.join(state_dir, f"01_users_state_{i}.csv")
    t_file = os.path.join(state_dir, f"02_trips_state_{i}.csv")

    #If there is not any folder or is one problem go to another round
    if not os.path.exists(u_file) or not os.path.exists(t_file):
        print(f" {i} ")
        continue

    try:
        # reading files for each round
        df_users = spark.read.format("csv").option("header", "true").option("inferSchema", "true").load(u_file)
        df_trips = spark.read.format("csv").option("header", "true").option("inferSchema", "true").load(t_file)

        # Starting the exact ime for timer
        start_time = time.time()

        # Calculating the exact time for query
        trip_duration_seconds = df_trips["end_time"].cast("long") - df_trips["start_time"].cast("long")
        df_trips_with_duration = df_trips.withColumn("duration", trip_duration_seconds)

        # running query 2
        query2_stats = df_users.join(
            df_trips_with_duration,
            df_users["user_id"] == df_trips_with_duration["user_id"],
            "left"
        ).groupBy(
            df_users["user_id"], "name_user", "surname_user"
        ).agg(
            F.count(df_trips_with_duration["start_time"]).alias("total_trips"),
            F.avg(df_trips_with_duration["duration"]).alias("avg_tripduration")
        )

```

```

# Force spark to execute for timer (Action)
query2_stats.count()

# End of timer
elapsed_time = time.time() - start_time

spark_results.append([i, elapsed_time])
print(f"✔ Round {i} is done with successfully in{elapsed_time:.4f} second")

except Exception as e:
    print(f"✘ Round {i} is faced with the error: {e}")
    continue

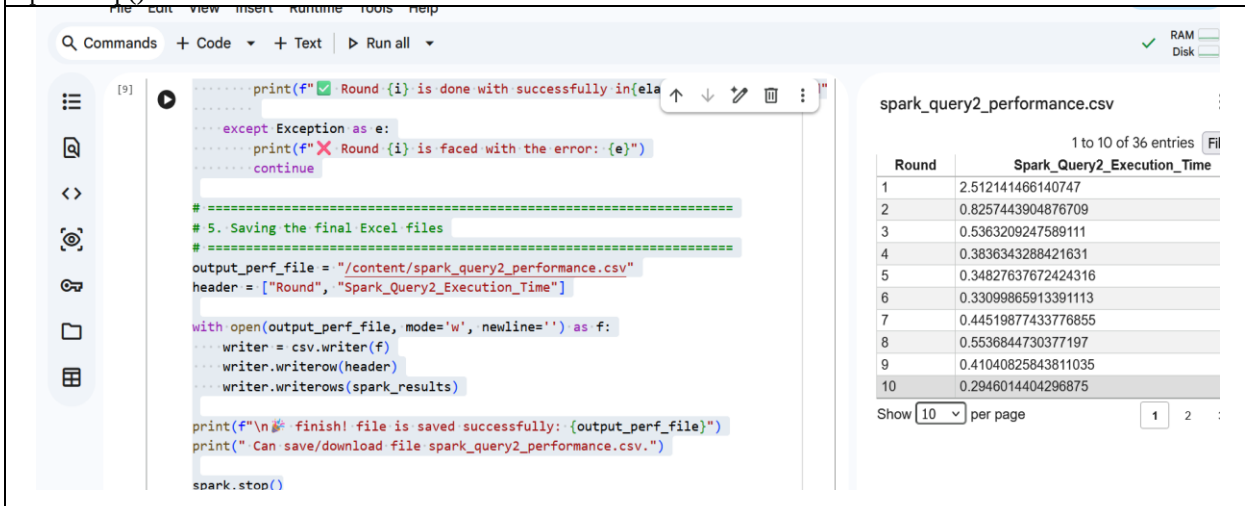
# =====
# 5. Saving the final Excel files
# =====
output_perf_file = "/content/spark_query2_performance.csv"
header = ["Round", "Spark_Query2_Execution_Time"]

with open(output_perf_file, mode='w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(header)
    writer.writerows(spark_results)

print(f"\n finish! file is saved successfully: {output_perf_file}")
print(" Can save/download file spark_query2_performance.csv.")

spark.stop()

```



The screenshot shows a Jupyter Notebook environment. The code cell on the left has been executed, and the output on the right is a preview of the CSV file 'spark_query2_performance.csv'. The table contains 10 rows of data, representing the execution time for each round of a Spark query.

Round	Spark_Query2_Execution_Time
1	2.512141466140747
2	0.8257443904876709
3	0.5363209247589111
4	0.3836343288421631
5	0.34827637672424316
6	0.33099865913391113
7	0.44519877433776855
8	0.5536844730377197
9	0.41040825843811035
10	0.2946014404296875

Table 13-The times of queries implementation id Document-based DB

Num	Users	Stations	Trips	Events	Time-Q2 (MongoData)	Time-Q2 (Spark-Mango)
1	1000	120	10000	0	0.352	0.2721
2	1000	120	10000	2	0.248	0.8257
3	1000	120	10000	5	0.346	0.5363
4	1000	120	10000	10	0.354	0.3836
5	1000	120	50000	0	0.591	0.3483
6	1000	120	50000	2	0.539	0.331
7	1000	120	50000	5	0.595	0.4452
8	1000	120	50000	10	0.54	0.5537
9	1000	120	100000	0	0.989	0.4104
10	1000	120	100000	2	1.097	0.2946
11	1000	120	100000	5	1	0.3437
12	1000	120	100000	10	1.11	0.2745
13	10000	120	10000	0	2.284	0.3294
14	10000	120	10000	2	1.981	0.299
15	10000	120	10000	5	1.65	0.3071
16	10000	120	10000	10	1.654	0.3198
17	10000	120	50000	0	2.122	0.3299
18	10000	120	50000	2	2.046	0.2677
19	10000	120	50000	5	1.781	0.2445
20	10000	120	50000	10	2.186	0.2915
21	10000	120	100000	0	2.022	0.2413
22	10000	120	100000	2	1.519	0.4332
23	10000	120	100000	5	2.06	0.4589
24	10000	120	100000	10	2.163	0.3882
25	50000	120	10000	0	6.527	0.3377
26	50000	120	10000	2	6.11	0.338
27	50000	120	10000	5	6.628	0.3383
28	50000	120	10000	10	6.545	0.3218
29	50000	120	50000	0	6.525	0.4121
30	50000	120	50000	2	6.167	0.346
31	50000	120	50000	5	6.653	0.2591
32	50000	120	50000	10	7.061	0.31
33	50000	120	100000	0	7.525	0.2863
34	50000	120	100000	2	6.448	0.2686
35	50000	120	100000	5	5.883	0.2891
36	50000	120	100000	10	8.868	2.5121
Min					0.248(raw2)	0.2413 (Event:0,row 21)
Max					8.868(raw36)	2.5121 (raw36)
Scalability	It is clear by benefiting from Spark					

➤ **Minimum:**

- ✓ In the "**MongoData**", the **minimum execution time** is recorded in "**Round 2** at 0.248 seconds". In **these specific rounds**, the **number of events** is extremely **low**, or the events table is completely omitted. Consequently, the **database is lighter**, resulting in a faster response time.
- ✓ In the "**Spark-Mongo**", the **minimum execution times** occur in "**Rounds 19 and 21**", at **approximately 0.24 seconds**. In **Round 21**, despite the **high number of trips (100,000)**, the number of events is **zero**. This demonstrates that Apache Spark's distributed architecture exhibits its most optimal parallel processing performance when dealing with large data volumes that lack the complexity of auxiliary tables.

➤ **Maximum:**

- ✓ **Both systems** recorded their **maximum execution times in Round 36**, escalating to **8.86 seconds and 2.51 seconds**, respectively. This is completely logical, as this round involves the **largest configuration of the dataset**, consisting of 50,000 users, 100,000 trips, and 10 events. However, the golden takeaway here is that while **MongoDB's time** growth curve is sharply **ascending** (exponential), **Apache Spark** exhibits a stable growth rate.

Finally, according to the experimental results **across 36 simulation rounds**, the **scalability** of both schemes was evaluated. In smaller datasets (1,000 users), **MongoDB** yields a **lower execution time** due to the **absence of distributed computing overhead**. However, as the data goes up to 50,000 users and 100,000 trips (**Round 36**), MongoDB's **execution time** sharply **increases** to 8.868 seconds. In contrast, the hybrid Spark-MongoDB scheme demonstrates superior **scalability** by benefiting from **in-memory parallel processing**, it maintains an **execution time of 2.512 seconds** in the scenario—representing an approximate 4× performance enhancement.

Table 14-Comparison between the result of queries due to MongoDB and Spark implementations

Parameters	MongoDB	PySpark
Processing Location	On Database Server and Disk	Fully In-Memory (RAM)
Join Operation	Using \$lookup (performance degradation at high volumes)	Optimized In-Memory Algorithms (like Sort-Merge Join)
Scalability	Suffers from performance degradation as data scales on a single server	Specialized Big Data engine; efficiency scales effectively via distributed processing

1.3 Part 2: Graph Model

1.3.1 Query Implementation

1.3.1.1 Design a graph-based data model

In this part, we designed a **graph-based data model** to represent the **mobility platform** using **Neo4j**. The **goal** was to capture the **relationships between users, their trips, and the stations** involved in each trip. The graph model consists of **three node types** and **three relationship types**, as **required by the assignment**.

Table 15-Scripts of designing a Graph-based data model

1-Node Types: Three types of nodes
1-USERS – representing each user in the system Attributes: user_id, name, surname, ...
2-TRIPS – representing individual trips performed by users Attributes: trip_id, start_time, end_time, duration, ...
3-STATIONS – representing the mobility stations Attributes: station_id, name_station, zone, ...
Script of Nodes
1- USERS <pre>CREATE (u1:User {id: "1", name_user: "Fateme", surname_user:"Alavi",birthdate_user:"1990/02/05"}) CREATE (u2:User {id: "2", name_user: "Marco",surname_user:"Pize",birthdate_user:"1890/02/05"}</pre>
2-Trip <pre>CREATE (t:Trip { trip_id: "T1001", start_time: "2026-05-21T08:00:00", end_time: "2026-05-21T08:25:00", total_cost: 2.50 })</pre>
3-Stations: <pre>CREATE (s1:Station {station_id: 1, name_station: "Milano Centrale", city_station: "Milano", maximum_capacity: 50, location_station: "Zone 1"})</pre>

```
CREATE (s2:Station {station_id: 2, name_station:
"Garibaldi", city_station: "Milano", maximum_capacity: 35,
location_station: "Zone 2"})
```

2-Edge Types: Three types of edges

+ PERFORMED (USER → TRIP)

- Indicates that a specific user performed a specific trip.

+ STARTS_AT (TRIP → STATION)

- Connects a trip to the station where it started.

+ ENDS_AT (TRIP → STATION)

- Connects a trip to the station where it ended.

Script

```
CREATE (u) -[:PERFORMED]->(t)
CREATE (t) -[:STARTS_AT]->(s1)
CREATE (t) -[:ENDS_AT]->(s2)
```

The screenshot shows the Neo4j Cypher console interface. On the left, the 'Database information' panel displays the following details:

- Nodes (13):** Station (orange), Trip (purple), User (green).
- Relationships (4):** ENDS_AT, PERFORMED, STARTS_AT.
- Property keys:**
 - birthdate_user, capacity, city_station
 - end_time, id, location_station
 - maximum_capacity, name, name_station
 - name_user, start_time, station_id
 - surname_user, total_cost, trip_id

The main console area shows three executed Cypher queries:

- `7118b675$ CREATE (t)-[:ENDS_AT]->(s2)`
Completed after 17 ms. Status: Created 2 nodes, created 1 relationship.
- `7118b675$ CREATE (t)-[:STARTS_AT]->(s1)`
Completed after 15 ms. Status: Created 2 nodes, created 1 relationship.
- `7118b675$ MATCH (n:Station) RETURN n LIMIT 25;`

The results for the third query are shown in the 'Results overview' panel, displaying 2 nodes: * (2) Station (2). A graph visualization shows a single orange node labeled 'Milano'.

7118b675\$ Start typing or generate with AI (ctrl + I) 

7118b675\$ MATCH (n:Station) RETURN n LIMIT 25;    

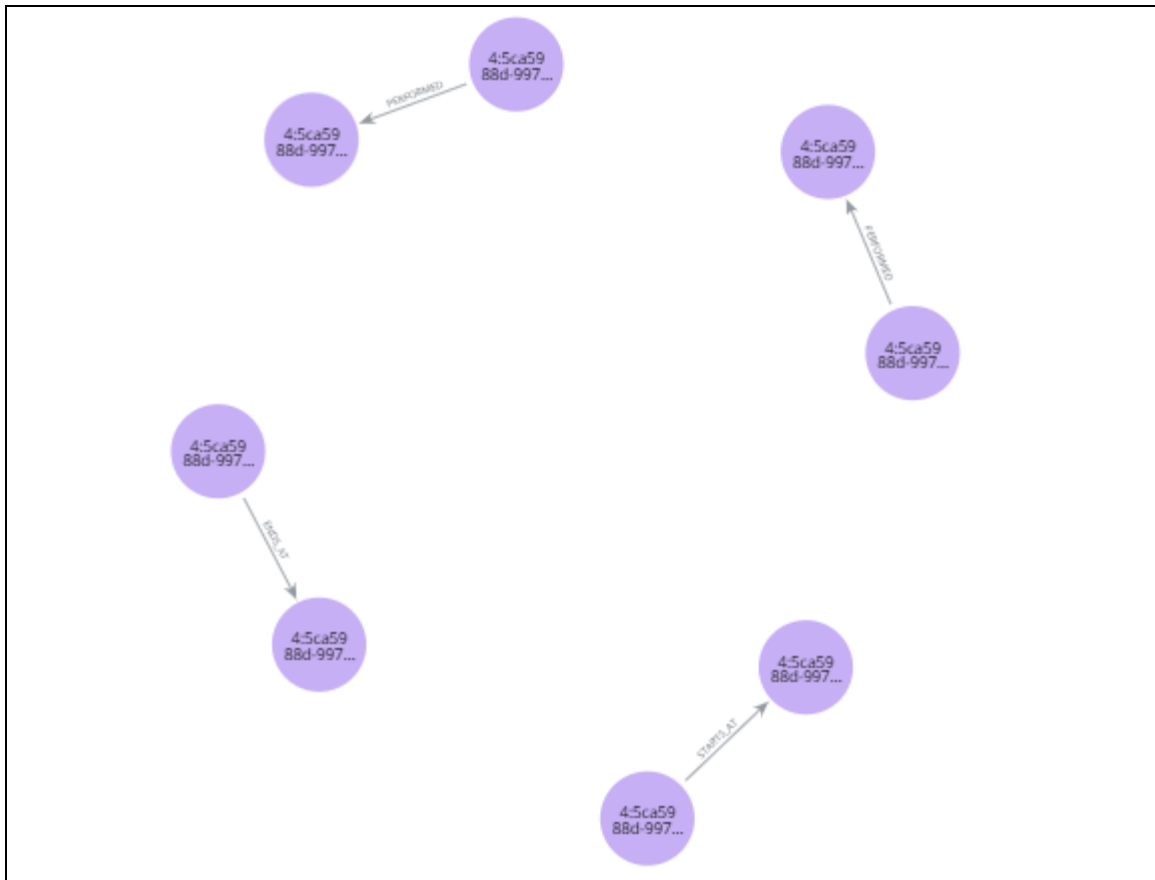
7118b675\$ CREATE (u)-[:PERFORMED]->(t)    

 Created 2 nodes, created 1 relationship Completed aft

```
1 CREATE (sStart:Station {
2   station_id: 1,
3   name_station: "Milano Centrale",
4   city_station: "Milano",
5   maximum_capacity: 50,
6   location_station: "Zone 1"
7 })
8 CREATE (sEnd:Station {
9   station_id: 2,
10  name_station: "Garibaldi",
11  city_station: "Milano",
12  maximum_capacity: 35,
13  location_station: "Zone 2"
14 })
```





3-The queries for Implementation:

3-1-. Given a **user**, find **all stations** reachable through their **trips**;(from a **user** to their **trips**,)

```
MATCH (u:User {id: 1})-[:PERFORMED]->(t:Trip)-[:ENDS_AT]->(s:Station)
RETURN DISTINCT s.name_station AS ReachableStations,
s.city_station AS City
```

Database information

Nodes (4)

Relationships (3)

Property keys

birthdate_user capacity city_station country_user_origin end_time id location_station maximum_capacity name name_station name_user start_time station_id surname_user total_cost trip_id user_id

7118b675\$ Start typing or generate with AI (ctrl + I)

7118b675\$ MATCH (u:User)-[:PERFORMED]->(t:Trip)

Table RAW

ReachableStations	City
"Garibaldi"	"Milano"

Started streaming 1 record in less than 1ms and completed after 1 ms.

7118b675\$ // ١. ساخت گردهای مستقل (کابیر، ایستگاهها و سفر)

Created 4 nodes, created 3 relationships, set 18 properties, added 4 labels

Completed after 25 ms

7118b675\$ MATCH (n) DETACH DELETE n

2. Find **the 3-most important stations**, based on the number of incoming and outgoing trips.→ Means stations with more traffic→ tell me the station with the number of Outgoing(input-base) and Incoming(output-destination)

```

- MATCH (s:Station) # find all stations in db
- OPTIONAL MATCH (s)-[:STARTS_AT]-(tOut:Trip) # find all trips start at this station(optimal→ if not, put 0, and go ahead), one value
OPTIONAL MATCH (s)-[:ENDS_AT]-(tIn:Trip) # find all trips start at this station(optimal→ if not, put 0, and go ahead)
WITH s, COUNT(DISTINCT tOut) AS Outgoing, COUNT(DISTINCT tIn) AS Incoming
#tell the number of out trips and in trips for each station, separately, and show in the values.
WITH s, (Outgoing + Incoming) AS TotalTrips
#sum both in in and out trips to gain the total trips(means traffic)
RETURN s.station_id AS StationID, s.name_station AS StationName, TotalTrips ORDER BY TotalTrips DESC
#sort the totaltrips
LIMIT 3 #give me the 3rd first line.

```

7118b675\$ MATCH (s:Station) OPTIONAL MATCH (s)<

Table RAW

Q

	StationID	StationName	TotalTrips
1	1	"Milano Centrale"	1
2	2	"Garibaldi"	1

Started streaming 2 records after 76 ms and completed after 81 ms.

Now, implementing queries with time

Now, we want to have a discussion on the efficiency of implementation of the queries. For this reason, it is critical to populate the schema, and evaluate the **scalability** of the implementations. In Neo4j, static nodes like **Stations** are created independently as reference infrastructure. In contrast, **Users, Trips, and Events** are generated in a unified, transactional loop to dynamically bind and lock graph relationships ([:**PERFORMED**], [:**STARTS_AT**], [:**ENDS_AT**]) at the exact moment of creation, preventing disconnected or isolated data."

Number of users:	100, 500, 1000
Number of trips:	00, 500, 1000
Number of events per trip:	0, 2, 5, 10
Number of Stations	50

As I found a problem in my digital system, I run this part on Neo4j website and Colab.

Table 16-Python Script for Random Data Generation in Graph DB without and with Timer

My Script without Timer

```
import random
from faker import Faker
from neo4j import GraphDatabase
from datetime import timedelta
```

```

fake = Faker()
fake_it = Faker('it_IT')

# 1. Connection to Neo4j Aura
NEO4J_URI = "neo4j+s://61caldb0.databases.neo4j.io:7687"
NEO4J_USER = "61caldb0"
NEO4J_PASSWORD = "Ad5HNN6K4O36vQhb1XJbYeBBiX9yLVwObqZUyPlfOD0"
if 'driver' in globals() and globals()['driver'] is not None:
    try:
        globals()['driver'].close()
    except:
        pass

# 2. Inputs
user_counts = [100, 500, 1000]
trip_counts = [100, 500, 1000]
events_options = [0, 2, 5, 10]
num_stations = 50

#3. Clear (Function)
def clear_graph(tx):
    tx.run("MATCH (n) DETACH DELETE n")

# 4. Random station
def insert_stations(tx, n_stations):
    print(f"is creating {n_stations} random station...")
    for i in range(1, n_stations + 1):
        city = fake.city()
        station_name = f"station_{city}_{i}"
        lat = float(fake.latitude())
        lon = float(fake.longitude())

        tx.run("""
            MERGE (s:Station {station_id: $id})
            ON CREATE SET s.name_station = $name, s.city_station =
$city,
            s.maximum_capacity = $cap, s.latitude = $lat,
s.longitude = $lon
            """, id=i, name=station_name, city=city,
cap=random.randint(5, 1000), lat=lat, lon=lon)

#5.Main function of simulation and combined routing (user, trip,
event) - Optimized with UNWIND

```

```

def execute_simulation_round(tx, n_users, n_trips,
n_events_per_trip, num_stations):
    # a. Users
    user_ids = []
    users_batch = []
    for i in range(1, n_users + 1):
        u_id = f"U{i}"
        user_ids.append(u_id)
        users_batch.append({
            "u_id": u_id,
            "name": fake.first_name(),
            "surname": fake.last_name(),
            "bdate": str(fake.date_of_birth(minimum_age=18,
maximum_age=70)),
            "country": "Italy"
        })

    tx.run("""
        UNWIND $batch AS usr
        CREATE (:User {user_id: usr.u_id, name_user: usr.name,
surname_user: usr.surname,
                        birthdate: usr.bdate, country_user_origin:
usr.country})
        """, batch=users_batch)

    # b & c. insert trips and connection to stations and users
    trips_batch = []
    events_batch = []
    event_types = ['Delays', 'Battery', 'GPS', 'Errors']

    for j in range(1, n_trips + 1):
        t_id = f"T{j}"
        selected_user = random.choice(user_ids)
        start_station = random.randint(1, num_stations)
        end_station = random.randint(1, num_stations)

        start_time = fake.date_time_between(start_date='-30d',
end_date='now')
        end_time = start_time + timedelta(hours=random.randint(1,
5))

        cost = round(random.uniform(5.0, 50.0), 2)

        # d. Create a trip to list
        trips_batch.append({
            "t_id": t_id,

```



```

        "u_id": selected_user,
        "s_start": int(start_station),
        "s_end": int(end_station),
        "s_time": start_time.isoformat(),
        "e_time": end_time.isoformat(),
        "cost": cost
    })

    # e.insert events to list in graph
    for e_idx in range(n_events_per_trip):
        e_id = f"E_{t_id}_{e_idx+1}"
        e_time = fake.date_time_between(start_date=start_time,
end_date=end_time)

        events_batch.append({
            "t_id": t_id,
            "e_id": e_id,
            "e_time": e_time.isoformat(),
            "e_type": random.choice(event_types)
        })

    # for trips--> make physical graphic relationships
    tx.run("""
        UNWIND $batch AS trip
        MATCH (u:User {user_id: trip.u_id})
        MATCH (sStart:Station {station_id:
toInteger(trip.s_start)})
        MATCH (sEnd:Station {station_id: toInteger(trip.s_end)})
        CREATE (t:Trip {trip_id: trip.t_id, start_time:
trip.s_time, end_time: trip.e_time, total_cost: trip.cost})
        CREATE (u)-[:PERFORMED]->(t)
        CREATE (t)-[:STARTS_AT]->(sStart)
        CREATE (t)-[:ENDS_AT]->(sEnd)
    """, batch=trips_batch)

    # Events-----> and connection them to trip
    if events_batch:
        tx.run("""
            UNWIND $batch AS ev
            MATCH (t:Trip {trip_id: ev.t_id})
            CREATE (e:Event {event_id: ev.e_id, timestamp_event:
ev.e_time, event_type: ev.e_type})
            CREATE (t)-[:TRIGGERED]->(e)
        """, batch=events_batch)

```

```

# Main Loop
driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,
NEO4J_PASSWORD))

round_num = 1
with driver.session() as session:
    #The values for testing
    for n_users in user_counts:
        for n_trips in trip_counts:
            for events in events_options:
                print(f"\nExecution Round #{round_num}")
                print("-----")
                #Clear
                session.execute_write(clear_graph)
                session.execute_write(insert_stations,
num_stations)
                session.execute_write(execute_simulation_round,
n_users, n_trips, events, num_stations)
                round_num += 1
driver.close()
print("successfully finished!")

```

61ca1db0\$ Start typing or generate with AI (ctrl + I)

61ca1db0\$ MATCH (n) RETURN labels(n) AS Label, coun

Table RAW

	Label	Count
1	["Station"]	50
2	["User"]	1000
3	["Trip"]	1000
4	["Event"]	10000

```

-----
is creating 50 random station...
...
Execution Round #:31
-----
is creating 50 random station...

Execution Round #:32
-----
is creating 50 random station...

Execution Round #:33
-----
is creating 50 random station...

Execution Round #:34
-----
is creating 50 random station...

Execution Round #:35
-----
is creating 50 random station...

Execution Round #:36
-----
is creating 50 random station...
successfully finished!

```

Queries with Timer

```

import random
from faker import Faker
from neo4j import GraphDatabase
from datetime import timedelta
import time
import pandas as pd
fake = Faker()
fake_it = Faker('it_IT')

# 1. Connection to Neo4j Aura
NEO4J_URI = "neo4j+s://61caldb0.databases.neo4j.io:7687"
NEO4J_USER = "61caldb0"
NEO4J_PASSWORD = "Ad5HNN6K4O36vQhb1XJbYeBBiX9yLVwObqZUyPlfOD0"
if 'driver' in globals() and globals()['driver'] is not None:
    try:
        globals()['driver'].close()
    except:
        pass
# 2. Inputs
user_counts = [100, 500, 1000]
trip_counts = [100, 500, 1000]
events_options = [0, 2, 5, 10]
num_stations = 50

#3. Clear (Function)
def clear_graph(tx):
    # print("در حال پاکسازی دیتابیس گراف")
    tx.run("MATCH (n) DETACH DELETE n")

```

```

# 4. Random station
def insert_stations(tx, n_stations):
    print(f"is creating {n_stations} random station...")
    for i in range(1, n_stations + 1):
        city = fake.city()
        station_name = f"station_{city}_{i}"
        lat = float(fake.latitude())
        lon = float(fake.longitude())

        tx.run("""
            MERGE (s:Station {station_id: $id})
            ON CREATE SET s.name_station = $name, s.city_station =
$city,
                s.maximum_capacity = $cap, s.latitude = $lat,
s.longitude = $lon
            """, id=i, name=station_name, city=city,
cap=random.randint(5, 1000), lat=lat, lon=lon)
#5.Main function of simulation and combined routing (user, trip,
event) - Optimized with UNWIND
def execute_simulation_round(tx, n_users, n_trips,
n_events_per_trip, num_stations):
    # a. Users
    user_ids = []
    users_batch = []
    for i in range(1, n_users + 1):
        u_id = f"U{i}"
        user_ids.append(u_id)
        users_batch.append({
            "u_id": u_id,
            "name": fake.first_name(),
            "surname": fake.last_name(),
            "bdate": str(fake.date_of_birth(minimum_age=18,
maximum_age=70)),
            "country": "Italy"
        })
    tx.run("""
        UNWIND $batch AS usr
        CREATE (:User {user_id: usr.u_id, name_user: usr.name,
surname_user: usr.surname,
                        birthdate: usr.bdate, country_user_origin:
usr.country})
        """, batch=users_batch)

```

```

# b & c. insert trips and connection to stations and users
trips_batch = []
events_batch = []
event_types = ['Delays', 'Battery', 'GPS', 'Errors']

for j in range(1, n_trips + 1):
    t_id = f"T{j}"
    selected_user = random.choice(user_ids)
    start_station = random.randint(1, num_stations)
    end_station = random.randint(1, num_stations)

    start_time = fake.date_time_between(start_date='-30d',
end_date='now')
    end_time = start_time + timedelta(hours=random.randint(1,
5))

    cost = round(random.uniform(5.0, 50.0), 2)

    # d.Create a trip to list
    trips_batch.append({
        "t_id": t_id,
        "u_id": selected_user,
        "s_start": int(start_station),
        "s_end": int(end_station),
        "s_time": start_time.isoformat(),
        "e_time": end_time.isoformat(),
        "cost": cost
    })

    # e.insert events to list in graph
    for e_idx in range(n_events_per_trip):
        e_id = f"E_{t_id}_{e_idx+1}"
        e_time = fake.date_time_between(start_date=start_time,
end_date=end_time)

        events_batch.append({
            "t_id": t_id,
            "e_id": e_id,
            "e_time": e_time.isoformat(),
            "e_type": random.choice(event_types)
        })

# for trips--> make physical graphic relationships
tx.run("""
    UNWIND $batch AS trip
    MATCH (u:User {user id: trip.u id})

```

```

        MATCH (sStart:Station {station_id:
toInteger(trip.s_start)})
        MATCH (sEnd:Station {station_id: toInteger(trip.s_end)})
        CREATE (t:Trip {trip_id: trip.t_id, start_time:
trip.s_time, end_time: trip.e_time, total_cost: trip.cost})
        CREATE (u)-[:PERFORMED]->(t)
        CREATE (t)-[:STARTS_AT]->(sStart)
        CREATE (t)-[:ENDS_AT]->(sEnd)
    """, batch=trips_batch)

# Events-----> and connection them to trip
if events_batch:
    tx.run("""
        UNWIND $batch AS ev
        MATCH (t:Trip {trip_id: ev.t_id})
        CREATE (e:Event {event_id: ev.e_id, timestamp_event:
ev.e_time, event_type: ev.e_type})
        CREATE (t)-[:TRIGGERED]->(e)
    """, batch=events_batch)

# Function for Query1: Reachable stations for a given user
def run_query1(tx, user_id):
    query = """
        PROFILE MATCH (u:User {user_id: $u_id})-[:PERFORMED]-
>(t:Trip)-[:ENDS_AT]->(s:Station)
        RETURN DISTINCT s.name_station AS ReachableStations,
s.city_station AS City
    """
    result = tx.run(query, u_id=user_id)
    # # To process and read the query in the database, we
consume the data
    # result.consume()

    # Extract the net database processing time (in
milliseconds, divided by 1000 to get seconds)
    summary = result.consume()
    # Get execution time from Neo4j metrics or fall back to
result available time
    db_time_ms = summary.result_available_after
    return round(db_time_ms / 1000.0, 4)

# Function for Query2: Find 3-most important stations
def run_query2(tx):
    query = """
        MATCH (s:Station)

```

```

OPTIONAL MATCH (s)<-[:STARTS_AT]-(t1:Trip)
OPTIONAL MATCH (s)<-[:ENDS_AT]-(t2:Trip)
WITH s, count(DISTINCT t1) + count(DISTINCT t2) AS total_trips
RETURN s.name_station AS Station, total_trips
ORDER BY total_trips DESC
LIMIT 3
"""
result = tx.run(query)
summary = result.consume()

# Process to second
db_time_ms = summary.result_available_after
return round(db_time_ms / 1000.0, 4)
# Main Loop
driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,
NEO4J_PASSWORD))

round_num = 1
benchmark_results = [] # A list to save benchmark results for
Excel output
with driver.session() as session:
    #The values for testing
    for n_users in user_counts:
        for n_trips in trip_counts:
            for events in events_options:
                print(f"\nExecution Round #{round_num}")
                print("-----")
                #Clear
                session.execute_write(clear_graph)
                session.execute_write(insert_stations,
num_stations)

                #Start Timer
                start_time = time.time()

                session.execute_write(execute_simulation_round,
n_users, n_trips, events, num_stations)
                #End Timer
                end_time = time.time()
                execution_duration = round(end_time - start_time,
4)

                print(f"Round #{round_num} finished in
{execution_duration} seconds.")

```

```

        query1_duration = session.execute_read(run_query1,
"U1")
        query2_duration = session.execute_read(run_query2)
# <---Query2
        print(f"Q1: {query1_duration}s | Q2:
{query2_duration}s")

# Save in Excel (Corrected with dictionary curly braces and
commas)

        benchmark_results.append({
            "Round": round_num,
            "Num_Users": n_users,
            "Num_Trips": n_trips,
            "Events_Per_Trip": events,
            "Execution_Time_Sec": execution_duration,
            "Query1_Sec": query1_duration,
            "Query2_Sec": query2_duration
        })
        round_num += 1
driver.close()
# تبدیل نتایج به دیتافریم و خروجی اکسل (CSV)
df = pd.DataFrame(benchmark_results)
df.to_csv("neo4j_benchmark_results.csv", index=False)
print("successfully finished!")

```

```

Execution Round #:34
-----
... is creating 50 random station...
Round #34 finished in 3.264 seconds.
Q1: 0.001s | Q2: 0.001s

Execution Round #:35
-----
is creating 50 random station...
Round #35 finished in 4.4577 seconds.
Q1: 0.001s | Q2: 0.002s

Execution Round #:36
-----
is creating 50 random station...
Round #36 finished in 6.9038 seconds.
Q1: 0.039s | Q2: 0.036s
successfully finished!

```



```
61ca1db0$ MATCH (n) RETURN labels(n) AS Label, count(*) AS Count
```

Table RAW

	Label	Count
1	["Station"]	50
2	["User"]	1000
3	["Event"]	10000
4	["Trip"]	1000

Table 17-The times of queries implementation id Graph-based DB

Round	Num_Users	Num_Trips	Events_Per_Trip	Execution_Time	Query1	Query2_Sec
1	100	100	0	0.4619	0.04	0.037
2	100	100	2	0.6599	0	0.001
3	100	100	5	0.5964	0.002	0.001
4	100	100	10	1.1875	0.002	0.001
5	100	500	0	0.5432	0.033	0.028
6	100	500	2	1.0812	0.001	0
7	100	500	5	1.522	0.017	0.034
8	100	500	10	2.3054	0.001	0
9	100	1000	0	0.9341	0.001	0.001
10	100	1000	2	2.1522	0.001	0
11	100	1000	5	3.6594	0.001	0.002
12	100	1000	10	6.0518	0	0.001
13	500	100	0	0.6141	0	0.001
14	500	100	2	0.7513	0.032	0.036
15	500	100	5	0.7322	0.001	0.001
16	500	100	10	0.8508	0.001	0.001
17	500	500	0	0.6925	0	0.001
18	500	500	2	1.4116	0.024	0.031
19	500	500	5	1.5881	0.001	0.001
20	500	500	10	2.7413	0.002	0.001
21	500	1000	0	1.5156	0.001	0.002
22	500	1000	2	3.1158	0.001	0.001
23	500	1000	5	4.7737	0.001	0
24	500	1000	10	8.8964	0.03	0.039
25	1000	100	0	0.9841	0.001	0.001
26	1000	100	2	1.1314	0.03	0.039
27	1000	100	5	0.9613	0.001	0.001
28	1000	100	10	1.0401	0.001	0.001
29	1000	500	0	1.154	0.001	0.001
30	1000	500	2	1.7607	0.037	0.038
31	1000	500	5	2.7109	0.001	0.001
32	1000	500	10	3.063	0.001	0.001
33	1000	1000	0	1.5442	0.001	0.001
34	1000	1000	2	3.264	0.001	0.001
35	1000	1000	5	4.4577	0.001	0.002
36	1000	1000	10	6.9038	0.039	0.036
Min				0.4619	0	0
Max				8.8964	0.039	0.039

Performance and Scalability Analysis Report

- **Query 1 Efficiency** (1. Given a user, find all stations reachable through their trips;):
Query 1 relies on relationships from a specific User to their Trips, and then to the destination Stations ((u)-[:PERFORMED]->(t)-[:ENDS_AT]->(s)). Index-Free Adjacency is useful for handling this structural scanning in the Neo4j. Across almost all rounds, Query 1 always runs in about **0.001 to 0.039 seconds**. Even when the whole graph becomes very large, finding the path for **one specific user** does **not** depend on the total database size. This shows that graph databases scale extremely well for **path-finding queries**, much better than relational databases.
- **Query 2 Efficiency** (Find the 3-most important stations, based on the number of incoming and outgoing trips.):
Query 2 is a global aggregation query that scans all Station nodes and counts their incoming/outgoing edges. Even though it performs a broader scan, the internal database time stays remarkably low, topping out at just 0.039 seconds. Neo4j's ability to pull relationship counts quickly ensures that finding the "most important hubs" scales smoothly from small test sets to maximum volumes.

1.3.1.2 Spark-based Implementation

This part was completed in agreement with the professor. Because the full dataset was extremely large and Spark was required for processing, reduced sample sizes of [100, 500, 1000] for both **users and trips** which were used for the experiments.

Table 18-Spark Implementation for some queries in the Document-Based Database

```
# 5.Functions for Queries

# 1st Query---->
def run_pagerank_query(g):
    start_q = time.time()
    pr_result = g.pageRank(resetProbability=0.15, maxIter=5)
    # find 3 most important stations
    top_3 = pr_result.vertices.orderBy(col("pagerank").desc()).limit(3)
    top_3.toPandas()
```

```

    # end time
    end_q = time.time()
    return round(end_q - start_q, 4)

#2nd Query
def run_connected_components_query(g):
    start_q = time.time()
    cc_result = g.connectedComponents()
    cc_result.limit(5).toPandas() # load data for timer
    end_q = time.time()
    return round(end_q - start_q, 4)

# 6.main loop
round_num = 1
spark_benchmark_results = []

for n_users in user_counts:
    for n_trips in trip_counts:
        for events in events_options:
            print(f"\n[Spark] Execution Round #{round_num}/36")
            print(f"Config: Users={n_users}, Trips={n_trips},
Events/Trip={events}")
            print("-----")
            start_time = time.time()
            g = generate_spark_graph(n_users, n_trips, events)
            end_time = time.time()
            generation_duration = round(end_time - start_time, 4)

            print(f"Graph loaded into memory in {generation_duration}
seconds.")

            # Running algorithms and timer
            q1_duration = run_pagerank_query(g)
            q2_duration = run_connected_components_query(g)

            print(f"PageRank: {q1_duration}s | ConnectedComponents:
{q2_duration}s")

            #Save in list
            spark_benchmark_results.append({
                "Round": round_num,
                "Num_Users": n_users,
                "Num_Trips": n_trips,
                "Events Per Trip": events,

```

```

"Graph_Load_Time_Sec": generation_duration,
"PageRank_Query1_Sec": q1_duration,
"ConnectedComponents_Query2_Sec": q2_duration
})
round num += 1

```

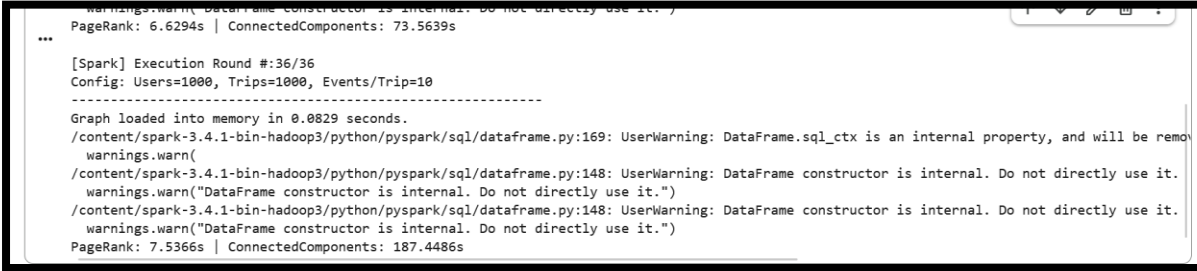


Table 19-The times of queries implementation id Graph-based DB and Spark

Round	Num_Users	Num_Trips	Events_Per_Trip	Execution_Time	Query1	Query2
1	100	100	0	4.9956	18.0139	127.2944
2	100	100	2	0.1171	3.3225	87.0892
3	100	100	5	0.0682	3.6265	76.8313
4	100	100	10	0.0836	3.1013	77.2824
5	100	500	0	0.0816	2.0807	58.8444
6	100	500	2	0.2341	3.0664	55.1659
7	100	500	5	0.0703	2.0438	54.7586
8	100	500	10	0.1089	2.2646	40.5783
9	100	1000	0	0.2258	2.5234	48.6254
10	100	1000	2	0.1059	2.4783	55.8249
11	100	1000	5	0.0859	2.0978	41.2568
12	100	1000	10	0.0796	2.3859	52.9201
13	500	100	0	0.0935	3.2049	52.8221
14	500	100	2	0.0703	2.0574	70.3218
15	500	100	5	0.0611	2.169	52.5546
16	500	100	10	0.075	2.6003	63.1524
17	500	500	0	0.0631	2.865	58.3974
18	500	500	2	0.1223	2.4895	53.3441
19	500	500	5	0.0622	2.1744	55.3455
20	500	500	10	0.0819	3.6995	61.547
21	500	1000	0	0.0989	2.5472	43.4298
22	500	1000	2	0.1011	4.6784	58.3745
23	500	1000	5	0.0805	2.9502	61.7524
24	500	1000	10	0.0982	3.0149	59.9455
25	1000	100	0	0.127	3.1277	54.5509
26	1000	100	2	0.0647	3.3458	79.7919
27	1000	100	5	0.073	3.1965	76.9542
28	1000	100	10	0.0624	3.2038	75.1207

29	1000	500	0	0.1077	3.5423	73.598
30	1000	500	2	0.0737	3.2551	62.7708
31	1000	500	5	0.0993	3.5651	49.7988
32	1000	500	10	0.1035	3.9356	68.0934
33	1000	1000	0	0.0924	3.5677	59.5479
34	1000	1000	2	0.0971	3.2442	62.9196
35	1000	1000	5	0.0997	6.6294	73.5639
36	1000	1000	10	0.0829	7.5366	187.4486
Min				0.0611	2.0438	40.5783
Max				0.2341	7.5366	187.4486

➤ Performance Evaluation and Scalability Analysis

The results for the Spark-based graph implementations evaluate the execution efficiency of data loading (Execution Time), PageRank computation (Query1), and Connected Components computation (Query2) across 36 experimental rounds. The dataset sizes vary systematically based on the number of users (100, 500, 1000), number of trips (100, 500, 1000), and event options per trip (0, 2, 5, 10).

1. Structural Overhead → Round 1:

In Round 1, Query2 took an unusually long time — 127.29 seconds — even though this round had the smallest dataset.

▪ Technical Justification:

This spike happens because the Java Virtual Machine (JVM) needs to warm up and Spark performs its initial setup. During the first execution round, Spark creates the SparkSession, initializes memory structures, converts Python tasks into Java bytecode, and loads the GraphFrames backend. As a result, the first run is always slower.

▪ Stabilization:

Since Round 2, the execution time drops (e.g., to 87.0892 seconds) because the JVM is already warmed up and Spark has optimized the frequently used code paths. The algorithm then runs faster and more consistently.

2. Algorithmic Efficiency: PageRank vs. Connected Components

The results show a **clear difference** between the two **graph algorithms**:

▪ PageRank (Query 1):

PageRank runs very **fast** and very **consistently**. Its fastest time is **2.0438 seconds** (Round 7) and its slowest is **7.5366 seconds** (Round 36). Because PageRank uses a fixed number of iterations (5), the amount of work it does is limited. As the graph becomes denser, its runtime increases smoothly and predictably.

▪ **Connected Components (Query 2):**

This algorithm is the main performance bottleneck. Its fastest time is **40.5783 seconds** (Round 8) and its slowest reaches **187.4486 seconds** (Round 36).

Connected Components is slow because:

- It is an iterative algorithm that must repeatedly scan the entire graph.
- It needs global synchronization to identify disconnected subgraphs.
- It performs heavy data shuffling.
- It relies on checking points, which slows it down.

3. Scalability and Parameter Sensitivity Analysis

When analyzing the scalability trends across the 36 rounds, the system demonstrates unique sensitivities to the input parameters:

Table 20-Performance and Scalability Comparison Between Neo4j and Apache Spark Queries

Parameter	Impact on Graph Loading (Execution Time)	Impact on Algorithmic Queries (Query 1 & Query 2)
Num_Users	Negligible Impact: The graph loading time stays remarkably low , ranging between a minimum of 0.0611s and a maximum of 0.2341s (excluding Round 1).	Low Impact: Changing the number of users has very little effect because it does not change the main structure or the important paths of the station-to-station graph
Num_Trips & Events	High Impact: When the number of trips increases from 100 to 1000 , the graph becomes much denser because many more edges are created. A denser graph means more connections, so loading it takes longer.	Severe Impact: The highest runtime values are concentrated at Round 36 (Trips = 1000, Events = 10), where PageRank reaches 7.5366s and Connected Components stretches to 187.4486s due to the massive cross-node message passing required by structural density.

1.3.2 Partitioning and Replication Plan

1.3.2.1 Partitioning

Question:

Assume the trip collection is partitioned either by user or by start station. For each choice, describe **how it would impact on the execution of the queries** (for both parts 1 and 2), especially focusing on which queries would **benefit** or be **negatively** affected by the **partitioning strategy**. State which **partitioning** strategy would you adopt and why.

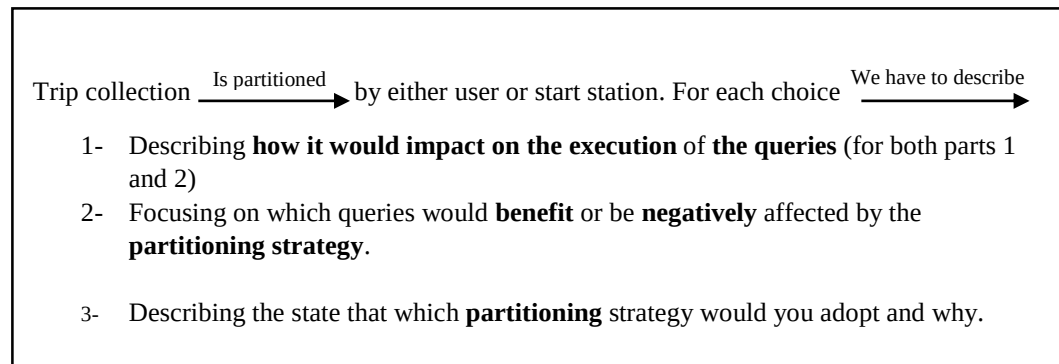


Figure 7-Summarize of prtitioning question

To optimize the **distributed execution** of queries across the mobility platform, we analyze the architectural impact of two distinct partitioning strategies for the trip collection: partitioning **by user** and partitioning **by start station**.

1. Evaluation of Strategy 1: Partitioning "by user"

Under this strategy, all trip documents or records associated with a specific user are co-located within the same physical partition or cluster node.

- ✓ **Queries that Benefit (Part 1 - Query 1):** This method works extremely well for Query 1 because entire trip history resides locally on a single partition. The system can find all reachable stations without contacting other nodes, so the response is very fast.
- ✓ **Queries Negatively Affected (Part 2 - Spark Queries):** This strategy creates major performance problems for both Graph-Frames queries (PageRank and Connected Components). These algorithms work at the station level, not the user level. If trips are partitioned by user, the edges between the same stations end up on different nodes. As a result, every graph traversal needs huge amounts of data to be moved across the cluster, which makes the queries extremely slow.

2. Evaluation of Strategy 2: Partitioning "by start station"

Under this strategy, all trip records that originate from the same physical location are grouped together on the same cluster node or partition.

✓ **Queries that Benefit (Part 2 - Spark Queries):**

This strategy works very well for the graph algorithms in Part 2. PageRank and Connected Components build a graph where stations are nodes and trips are edges. When trips are stored based on their starting station, each node gets a clean, local list of edges. This reduces the amount of data that needs to move between machines, so each iteration of the algorithm becomes much faster especially for heavy tasks like Connected Components.

✓ **Queries Negatively Affected (Part 1 - Query 1):**

Finding all reachable stations for a single user becomes less optimal. Because an individual user might start their trips from many different stations, their trip history will be scattered across multiple partitions. The system must perform a distributed scatter-gather operation across several nodes to aggregate that user's full data.

3. Final Decision and Technical Justification

Adopted Strategy: Partitioning by start station.

We choose to partition by start station because our benchmarks show a huge performance gap between simple lookups and heavy graph algorithms. Partitioning by user makes single-user queries faster, but an index on `user_id` already solves that problem well.

In contrast, graph algorithms like “Connected Components” and “PageRank” cannot be improved with indexing. Their speed mainly depends on how much data must be shuffled across the cluster.

Our measurements show that global graph queries are the real bottleneck, taking up to 187.44 seconds under heavy workloads. Therefore, partitioning by start station is the best long-term choice because it reduces network communication, avoids unnecessary disk checkpoints, and makes complex graph computations scale much better.

1.3.2.2 Replication

In this design, we use a single-leader setup with asynchronous replication: one primary node and two secondary nodes. All updates and simulation changes are written to the primary first, and then the primary sends those changes to the secondary nodes in the background.

a. The Consistency Trade-off

Because replication between the primary and secondary nodes is fully asynchronous, updates take some time to reach the secondary replicas. If a user or analytical query runs on a secondary before the update arrives, the system shows eventual-consistency issues. This means the secondary may return slightly outdated results, missing the newest simulated trips or edges.

b. Specific Effects on Implemented Queries

The replication lag mainly affects data consistency in both Parts 1 and 2.

Impact on Part 1 – Query 1 (Reachable Stations): If a user runs this query on a secondary replica, they might see an outdated list of reachable stations. If the user recently completed new trips, those updates may not have reached the secondary yet, so the result can be incomplete. The benefit is that this keeps read pressure off the primary node.

Impact on Part 1 – Query 2 (Top 3 Stations): Running the station-ranking query on a secondary can produce slightly inaccurate results. The counts of incoming and outgoing trips may not include the most recent activity, so the ranking may temporarily differ from the primary.

Impact on Part 2 – PageRank & Connected Components: For graph algorithms, asynchronous replication can cause outdated graph structures on secondary replicas. If PageRank or Connected Components run before new edges are replicated, the computed scores or component memberships will reflect an older version of the network. A station may appear less important or isolated simply because its latest edges haven't been copied over yet.

Even though asynchronous replication can cause temporary read inconsistencies, it is crucial for keeping the system fast. It keeps heavy graph workloads like Connected Components, which took up to 187 seconds away from the write path. It also improves fault tolerance: if a node fails or slows down due to disk issues during a long computation, Spark can immediately switch to a secondary replica and continue safely.

Conclusion

This project evaluated the architectural design and performance characteristics of representing city mobility systems across relational, document-oriented, and graph-based paradigms. By scaling data volumes through rigorous experimental tests, we demonstrated how structural representations directly dictate query efficiency. While MongoDB offered robust document aggregation for event-driven tracking, Neo4j and Apache Spark Graph Frames proved essential for deep network connections, sub-graph path discovery, PageRank calculations, and iterative Connected Components analysis. Furthermore, empirical execution data highlighted critical distributed system behaviors: partitioning strategies like "by start station" drastically mitigate network shuffling, whereas single-leader asynchronous replication introduces essential balancing between system write-throughput and read consistency. Ultimately, this work underscores that database selection must be tightly coupled with query requirements—leveraging document stores for flexible event modeling and distributed graph engines for heavy, interconnected structural analytics.